System Domain - HanziApp

The HanziApp System Domain defines three things: the system context (what is inside, what is outside, and which external systems it talks to), the characterization of its actors, and the business rules that govern learning.

It builds on the requirements and user stories of Deliverable 1 (September 30, 2026), and adds the design decisions made after that deliverable:

- A deck can belong to several lists.
- A user can hold several roles; teachers also study.
- A flashcard's meaning is revealed step by step.
- Senses and Chinese example sentences are shared across languages; only their translations change.
- Each course appears to its students as a read-only list.

Identifiers keep their original prefixes so they match the requirements document: RF (functional requirement), HU (user story), RN (business rule) and RNF (non-functional requirement). RN-01 to RN-16 keep their numbers, new rules continue from RN-17, and a rule split into parts uses decimals (RN-15.1, RN-15.2).

1. System context

HanziApp is a responsive web application that helps Spanish and English speakers memorize the vocabulary and handwriting of HSK 3.0, levels 1 to 6. It interacts with four human actors and a time-based scheduler, and depends on six external systems. The advanced band of HSK 3.0 (levels 7 to 9) is out of scope.

1.1 Purpose

The system solves three problems for a Chinese learner:

- **What to study:** it offers the official HSK 3.0 vocabulary, organized into lists and decks.
- **When to review:** it schedules every word with FSRS and every character with an in-house motor-memory algorithm.
- **How to keep the habit:** it tracks the daily streak, learned words and characters, and progress per level.

It also gives teachers a way to assign vocabulary to their courses and follow the group's progress.

1.2 Domain glossary

These terms are used throughout the System Domain and the other deliverables.

Term	Meaning in HanziApp
HSK 3.0	The new Chinese Proficiency Test, based on the official syllabus released in 2025 and applied from 2026. It has 9 levels in three bands (elementary 1–3, intermediate 4–6, advanced 7–9); HanziApp covers levels 1 to 6.
Hanzi	A Chinese character, such as 好.
Word	A vocabulary item made of one or more hanzi with a specific reading, such as 你好 (nǐ hǎo).
Pinyin	The official romanization of Mandarin. It marks tones with accents (hǎo) or with numbers (hao3).
Tone	The pitch pattern of a syllable. Mandarin has four tones and a neutral tone, and a different tone can change the meaning.
Sense	One meaning of a word, with its part of speech. 还 (hái) has several senses, such as "still" and "also".
Stroke order	The standard order and direction in which the strokes of a character are written.
Deck	A group of words studied together, such as "Greetings".
List	A group of decks. System lists are HSK 1 to HSK 6; users can also create their own.
Active recall	Retrieving an answer from memory before seeing it, which is how a flashcard works.

Spaced repetition	A study technique that schedules each review just before the learner is likely to forget the item.
FSRS	Free Spaced Repetition Scheduler: the algorithm that computes each word's next review date.
Stability	In FSRS, the number of days until the probability of remembering a word falls to 90%.
Due review	A word or character whose next review date has arrived.
Motor memory	The memory of how to write a character by hand, trained in the writing practice.
Streak	The number of consecutive days on which the user completed at least one study session.

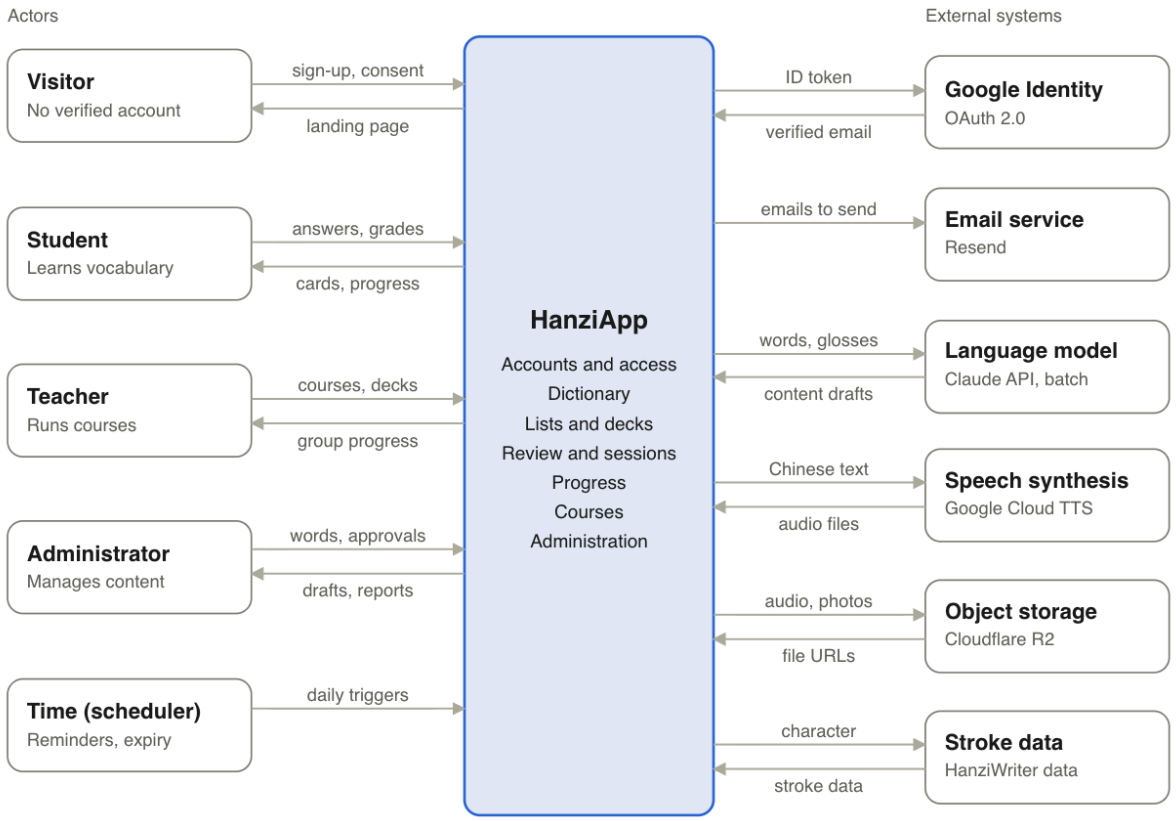
1.3 System boundary

Seven subsystems sit inside the system:

1. Accounts and access
2. Content and dictionary
3. Lists and decks
4. Review and study sessions (flashcards and writing)
5. Progress and motivation
6. Courses
7. Content and user administration

Google identity, email delivery, AI content generation, speech synthesis, file storage and stroke data are handled by external systems.

HanziApp exchanges data with five actors and six external systems



Each arrow is labeled with the data it carries.

Each arrow is labeled with the data it carries: arrows pointing into HanziApp are inputs and arrows pointing out are outputs.

1.4 External systems

External system	What it exchanges with HanziApp	Direction	Status	RF
Google Identity (OAuth 2.0)	The user's verified identity: email and name	Inbound	In use	RF-07, RF-08, RF-09
Transactional email service (Resend)	Verification, password reset, confirmation, request decision and reminder emails	Outbound	In use	RF-03, RF-05, RF-11, RF-22, RF-82, RF-87

Language model (Claude API, batch)	Receives words with their official gloss; returns senses, examples and translations as drafts	Both	In use	RF-102
Speech synthesis (Google Cloud TTS)	Receives Chinese text; returns audio for words and examples	Both	Planned	RF-37, RF-38
Object storage (Cloudflare R2)	Stores and serves audio files and profile photos	Both	Planned	RF-16, RF-37, RF-38
Character stroke data (HanziWriter)	Shape and order of each character's strokes	Inbound	Planned	RF-39, RF-61 to RF-65

The official HSK 3.0 vocabulary is not a connected system: it enters as a file through bulk import (RF-100).

The language model sits outside the boundary even though the generation pipeline currently runs as scripts. RF-102 moves the trigger into the administration module, so HanziApp calls the model the same way it calls any other external system.

1.5 Scope

In scope: the Must, Should and Could functional requirements of Deliverable 1. The first release of the semester commits to the Must requirements.

Out of scope:

- The HSK 3.0 advanced band, levels 7 to 9.
- Pronunciation recording and assessment (RF-85 and RF-86, Won't). They are documented as vision.
- Mock exams or HSK-style quizzes.

2. Actor characterization

HanziApp has four human actors and seven secondary actors. Human roles are cumulative: every verified account is a student account, and the teacher and administrator roles are added on top of it.

2.1 Human actors at a glance

Actor	Type	Who it is	How the role is obtained
Visitor	Primary	A person without an account, or with an account pending verification	Not a stored role: it is the absence of a verified account
Student	Primary	A user with a verified email who is learning Chinese	By verifying the email, or by signing up with Google
Teacher	Primary	A Chinese teacher (Confucius Institute, university programs) who can also study	An administrator approves their request (RF-87, RF-114)
Administrator / content manager	Primary	A HanziApp team member responsible for content and accounts	Another administrator assigns it (RF-112)

2.2 Actor generalization

Teacher and Administrator specialize Student: they inherit every student interaction and add their own. A visitor becomes a student once the email is verified.

[embed: node/6d589f32-d84b]

2.3 Actor profiles

The profile describes each human actor along the same dimensions. Values are the team's expected profile and will be validated with the onboarding questionnaire (RF-27 to RF-30).

Dimension	Visitor	Student	Teacher	Administrator

Technical skill	Basic	Basic; not a technical user (RNF-37)	Basic to intermediate	High; a member of the project team
Chinese level	Any, often a beginner	HSK 1 to 6; the first release has HSK 1 content	Advanced or native	Enough to check content against the official glossary
Typical use	One visit to evaluate the app and sign up	Short, frequent sessions, ideally every day	Around their classes, to assign decks and check progress	In batches, when loading or reviewing content
Main device	Phone (RNF-38)	Phone or computer; writes with a finger or a mouse (RNF-38, RNF-39)	Computer	Computer
Language	Spanish or English	Spanish or English, as chosen at sign-up	Spanish or English	Both, since they review content in both languages
Motivation	Find a tool to prepare for the HSK	Pass an HSK exam, study or travel in China, personal interest	Keep students practicing between classes	Publish correct content
Expected number	Every newcomer	The largest group; version 1 had 16 users from the Confucius Institute	A few per institution	The 5-person project team
Release 1 priority	Must	Must	Could	Must for content management

2.4 Visitor

- **Goals:** learn about the value proposition, create an account and verify the email.
- **Interactions:** landing page and language selector (RF-24, RF-25); sign-up with email or Google (RF-01, RF-02, RF-07, RF-14); email verification and resend (RF-04, RF-05); sign-in and password reset (RF-08, RF-10 to RF-12).
- **Constraints:** cannot use any student feature (RN-16.1). Must accept the data policy before the account is created (RN-17.1). A minor needs authorization from a legal guardian (RNF-33).

2.5 Student

- **Goals:** memorize the vocabulary and handwriting of their HSK level, keep a daily habit and know how ready they are for the exam.
- **Interactions:**
    - Profile and onboarding questionnaire (RF-15 to RF-23, RF-27 to RF-30).
    - Dictionary (RF-31 to RF-42).
    - Lists and decks (RF-43 to RF-53).
    - Flashcards, writing and sessions (RF-54 to RF-74).
    - Progress and motivation (RF-75 to RF-84).
    - Courses: enroll, leave and request the teacher role (RF-87, RF-90, RF-95).
- **Constraints:** cannot modify system or course lists and decks (RN-13). Only sees content approved in their study language (RN-14, RN-15.3). A suspended account cannot sign in (RN-22.1). Must finish a first session in under 3 minutes without help (RNF-37).

2.6 Teacher

- **Goals:** assign the vocabulary of their classes to their course and spot the students who need support.
- **Interactions:** course creation and invitation code regeneration (RF-88, RF-89); course decks and their assignment (RF-91, RF-92); individual and group progress (RF-93, RF-94). Keeps every student interaction.
- **Constraints:** only sees students enrolled in their courses (RN-47.1), and never their email or questionnaire answers (RN-47.2). If the role is removed, their courses become inactive (HU-112, RN-55).

2.7 Administrator / content manager

- **Goals:** publish correct content in both languages, maintain the system HSK lists and support users.
- **Interactions:**
    - Vocabulary: create, edit, deactivate and import (RF-96 to RF-101).
    - Content: generate with AI, validate, edit, approve and reject drafts (RF-99, RF-102 to RF-106).
    - User error reports (RF-107).
    - System decks (RF-108 to RF-110).
    - Users: search, roles, suspension and teacher requests (RF-111 to RF-115).

    - Usage metrics (RF-116).
- **Constraints:** their actions are recorded in the audit log (RN-49). Cannot remove the last administrator role nor suspend their own account (RN-18.3, RN-22.3). Cannot see passwords or questionnaire answers (HU-111), and metrics never identify an individual (HU-116).

2.8 Secondary actors

Actor	Type	Role in the system
Google Identity	External system	Authenticates the user and provides their verified email
Transactional email service	External system	Delivers emails in the user's study language
Language model	External system	Generates content drafts; never publishes (RN-14)
Speech synthesis	External system	Generates audio once (RNF-58)
Object storage	External system	Stores audio files and profile photos
Stroke data	External system	Provides the shape and order of each character's strokes
Time (scheduler)	Time actor	Sends reminders at 8:00 p.m. local time (RF-82), closes the day for the streak (RN-11), expires verification and reset links, and triggers the daily backup (RNF-10)

3. Business rules

HanziApp is governed by 96 business rules in eight categories, and each rule states exactly one thing. A rule from an earlier version that stated several things is split into numbered

parts (RN-15.1, RN-15.2…), so references to the original number stay valid. Three rules were added in this version to close gaps in data protection and courses (RN-53 to RN-55). RN-20, RN-21 and RN-35 were retired because they are security or interface requirements, not business rules (section 4.1).

Each rule is classified with the usual business-rule taxonomy:

Type	Meaning
Fact	A statement about the domain's structure or identity that is always true
Constraint	Something that must or must not happen; the system rejects anything that breaks it
Action enabler	A condition that automatically triggers an action
Inference	A conclusion the system derives from other facts
Computation	How a value is calculated

3.1 Accounts and access

ID	Rule	Type	Change	Source
RN-16.1	An account created with email and password stays pending, and cannot use any student feature until the email is verified.	Constraint	Deliverable 1	RF-03 to RF-06
RN-16.2	An email verification link expires 24 hours after it is sent.	Constraint	Deliverable 1	RF-04

RN-16.3	An account created with Google is considered verified from the moment it is created.	Inference	Deliverable 1	RF-07
RN-17.1	No account is created, with email or Google, without explicit acceptance of the current data policy.	Constraint	New	RF-02, RNF-26
RN-17.2	Accepting the data policy records the date, the time and the version accepted.	Action enabler	New	RNF-26
RN-18.1	Roles are cumulative: every verified account has the student role, and the teacher and administrator roles are added to it.	Fact	New	Design decision
RN-18.2	Removing a role never deletes the user's progress.	Constraint	New	HU-112
RN-18.3	The administrator role cannot be removed from the last administrator.	Constraint	New	HU-112
RN-19.1	One email maps to exactly one account.	Constraint	New	HU-07
RN-19.2	Signing in with Google with an email that is already registered links both access methods to that account and marks it as verified.	Action enabler	New	HU-09
RN-22.1	A suspended account cannot sign in by any method.	Constraint	New	HU-113

RN-22.2	Suspending an account ends all its active sessions.	Action enabler	New	HU-113
RN-22.3	Nobody can suspend their own account.	Constraint	New	HU-113
RN-22.4	Reactivating a suspended account keeps all its progress.	Constraint	New	HU-113
RN-23.1	Deleting an account permanently removes its personal data, lists, decks, progress and enrollments.	Action enabler	New	HU-22
RN-23.2	Signing up again with the email of a deleted account creates a new account with no progress.	Inference	New	HU-22
RN-53	When a new version of the data policy is published, existing users must accept it at their next sign-in before they can continue.	Constraint	New	Ley 1581 de 2012, RNF-26
RN-54	Every user can consult and correct their personal data and request its deletion at any time; deletion follows RN-23.1.	Constraint	New	Ley 1581 de 2012 (art. 8), RF-15, RF-22

3.2 Content

ID	Rule	Type	Change	Source
RN-14	No AI-generated content is visible to users until an administrator has reviewed and approved it.	Constraint	Deliverable 1	RF-102 to RF-105

RN-15.1	Each user has one study language, Spanish or English, chosen at sign-up.	Fact	Deliverable 1	RF-14
RN-15.2	The study language sets the language of the interface, senses, translations, meaning search and emails.	Inference	Deliverable 1	RF-17
RN-15.3	A word is shown in a language only when its content is approved in that language.	Constraint	Deliverable 1	RF-17
RN-15.4	Changing the study language does not affect progress.	Constraint	Deliverable 1	RF-17
RN-24	The hanzi, the pinyin, the senses with their part of speech, and the Chinese example sentences are shared across languages; only the translations change per language.	Fact	New	Design decision
RN-25.1	Each word's content has one status per language: pending, draft, approved or rejected.	Fact	New	RF-99
RN-25.2	There is at most one approved content per word and language.	Constraint	New	HU-105
RN-25.3	A draft with validation failures cannot be approved.	Constraint	New	HU-105, HU-106
RN-26.1	A word is identified by its hanzi and its pinyin, so the same hanzi with different readings, such as 还 (hái and huán), are different words.	Fact	New	HU-31, HU-96

RN-26.2	Each word belongs to exactly one HSK 3.0 level, from 1 to 6.	Constraint	New	HU-96
RN-27.1	A deactivated word does not appear in the dictionary, decks or sessions, and does not count in any metric.	Constraint	New	HU-98
RN-27.2	Deactivating a word keeps every user's progress on it, and that progress counts again if the word is reactivated.	Constraint	New	HU-98
RN-28	Each word is made of one or more characters in a fixed order, and one character can be part of many words.	Fact	New	RF-40, RF-65

3.3 Lists and decks

ID	Rule	Type	Change	Source
RN-03	Decks and lists only group words: deleting a deck or a list, or removing a word from a deck, never deletes the user's progress on that word or its characters.	Constraint	Deliverable 1	RF-46, RF-50, RF-51
RN-13	System lists (HSK 1 to 6) are read-only for students and teachers; only administrators can modify them.	Constraint	Deliverable 1	RF-43, RF-108 to RF-110
RN-29.1	A personal deck can belong to several of its owner's lists, with a separate position in each list.	Fact	New	Design decision

RN-29.2	Deleting a list also deletes the decks that belong to no other list.	Action enabler	New	Design decision, HU-46
RN-29.3	Deleting a deck removes it from every list it belongs to.	Action enabler	New	HU-50
RN-30.1	A list name is unique among its owner's lists.	Constraint	New	HU-44
RN-30.2	A deck name is unique within each list and within each course.	Constraint	New	HU-47, HU-91
RN-31.1	A system list contains only words of its own HSK level.	Constraint	New	HU-108, HU-109
RN-31.2	Each word of a system list is in exactly one of its decks.	Constraint	New	HU-109
RN-31.3	An empty system deck is not shown to users.	Constraint	New	HU-31
RN-32	Copying a system deck creates an independent personal deck: later changes to the original do not affect the copy.	Constraint	New	HU-52
RN-33.1	Each course appears to its enrolled students as a read-only list containing the assigned decks.	Inference	New	Design decision, HU-92
RN-33.2	The teacher's changes to an assigned deck are visible to students immediately.	Inference	New	HU-92

RN-33.3	Leaving a course removes the course list from the student's lists.	Action enabler	New	HU-95
RN-34	A study session cannot be started on an empty deck.	Constraint	New	HU-47

3.4 Review and writing

ID	Rule	Type	Change	Source
RN-01	Each user has a single FSRS review state per word (stability, difficulty and next review date), independent of the deck where the word is reviewed.	Constraint	Deliverable 1	RF-57
RN-02	Each user has a single writing review state per character, independent of the word where it is practiced and of the recognition state.	Constraint	Deliverable 1	RF-66
RN-05.1	Grading a card Easy (remembered) records an FSRS success, and its interval grows.	Computation	Deliverable 1	RF-56
RN-05.2	Grading a card Hard (not remembered) records an FSRS lapse, and its interval resets.	Computation	Deliverable 1	RF-56
RN-36.1	A card graded Hard comes back after at least 3 other cards, or immediately if it is the only one left.	Action enabler	New	HU-58

RN-36.2	After three Hard grades in the same session, a card leaves the session and stays due for the next day.	Action enabler	New	HU-58
RN-37.1	The new-word limit counts the new words studied per day, in the user's time zone, across all their decks.	Computation	New	RF-59
RN-37.2	The new-word limit defaults to 10 and accepts values from 0 to 50.	Constraint	New	RF-21
RN-37.3	Words seen in free review do not count toward the new-word limit.	Constraint	New	RF-59, RF-69
RN-38.1	The next writing review is computed from the errors, the hints used and the writing mode.	Computation	New	RF-66
RN-38.2	With the same performance, the next interval is shorter in easy mode than in hard mode.	Computation	New	HU-62, HU-66
RN-38.3	Marking "I don't know" resets the character's writing interval.	Action enabler	New	HU-66
RN-39	In hard mode, three consecutive errors on the same stroke show a hint for that stroke, and the hint counts in the next-review calculation.	Action enabler	New	RF-64

3.5 Study sessions

ID	Rule	Type	Change	Source

RN-04.1	A scheduled session's queue is built when it starts: first the deck's words due today or earlier, then new words up to the daily limit.	Computation	Modified	RF-68
RN-04.2	A session stores its queue so it can be resumed.	Constraint	Modified	RF-72
RN-04.3	Reviewing a word in any deck removes it from the due reviews of every deck.	Inference	Modified	RF-68
RN-04.4	A word already studied is never presented as new in another deck.	Constraint	Modified	RF-68
RN-12.1	An abandoned session stays in progress with its position: the current card and, in writing, the current character.	Constraint	Deliverable 1	RF-72
RN-12.2	There is at most one in-progress session per deck and practice type.	Constraint	Deliverable 1	RF-73
RN-12.3	A scheduled session is recalculated when it is resumed, while a free review keeps its order until it is finished or discarded.	Action enabler	Deliverable 1	RF-72, RF-73
RN-40	Free review presents every word of the deck in random order and does not change any review state.	Constraint	New	RF-69
RN-41	The all-due-reviews session gathers the due reviews from all the user's decks, each word once and with no new words.	Computation	New	RF-70

RN-42.1	Each grade and each completed character is saved on the server immediately.	Action enabler	New	RF-71
RN-42.2	Discarding a session does not undo the grades already recorded.	Constraint	New	HU-73
RN-43	Deleting a deck or a list, or leaving a course, discards the in-progress sessions of the affected decks.	Action enabler	New	HU-46, HU-50, HU-95

3.6 Progress and motivation

ID	Rule	Type	Change	Source
RN-06	A word is learned when its FSRS stability is 21 days or more.	Inference	Deliverable 1	RF-77
RN-07	A character is learned when its writing interval is 21 days or more and it has been practiced in hard mode.	Inference	Deliverable 1	RF-78
RN-08	Progress in an HSK level is the percentage of learned words over the active words of that level.	Computation	Deliverable 1	RF-79
RN-09	Metrics count distinct words and characters, without duplicates, even when they appear in several decks.	Computation	Deliverable 1	RF-77 to RF-80
RN-10.1	An active day is a calendar day, in the user's time zone, on which they complete at least one session of any scope, even with the same deck as on previous days.	Inference	Modified	RF-75

RN-10.2	A session counts for the day it is completed, and an abandoned session does not count.	Constraint	Modified	RF-75
RN-10.3	The active day is stored when the session completes and is not recalculated if the user changes time zone.	Action enabler	Modified	HU-19
RN-11	The streak is the number of consecutive active days, and it resets to zero when a day passes with no activity.	Computation	Deliverable 1	RF-75
RN-44	The longest streak is the highest value the streak has reached, and it never decreases.	Computation	New	RF-76
RN-45	Each badge is awarded at most once per user, even if the milestone is reached again.	Constraint	New	HU-84

3.7 Courses

ID	Rule	Type	Change	Source
RN-46.1	Enrolling requires a valid invitation code and agreeing to share progress with the teacher.	Constraint	New	HU-90
RN-46.2	Regenerating a course's code invalidates the previous one without affecting students already enrolled.	Action enabler	New	HU-89
RN-47.1	A teacher only sees the progress of students enrolled in their courses: streak, active days,	Constraint	New	RNF-32, HU-93

	learned words and progress on assigned decks.			
RN-47.2	A teacher never sees a student's email or questionnaire answers.	Constraint	New	HU-93
RN-47.3	A teacher stops seeing a student's progress once the student leaves the course.	Constraint	New	HU-95
RN-48.1	A user can have at most one pending teacher-role request.	Constraint	New	HU-87
RN-48.2	A teacher-role request must state the institution.	Constraint	New	HU-87
RN-55	An inactive course accepts no new enrollments and cannot be edited; its enrolled students keep the course list and can still study its decks.	Constraint	New	HU-112

3.8 Administration

ID	Rule	Type	Change	Source
RN-49	The audit log records, with author and date, every creation, edit and deactivation of a word; every approval and rejection of content; every system deck created; every role change and suspension; and every decision on a teacher request.	Action enabler	New	RNF-24
RN-50	Rejecting content, suspending an account and rejecting a teacher request each require a reason.	Constraint	New	HU-106, HU-113, HU-115

RN-51.1	AI generation works in batches of up to 100 words in one language.	Constraint	New	HU-102
RN-51.2	A generation batch skips the words that already have approved content in its language.	Constraint	New	HU-102
RN-52.1	An import never creates a duplicate word, that is, one with the same hanzi and pinyin.	Constraint	New	HU-100
RN-52.2	An import keeps the valid rows even when other rows fail.	Constraint	New	HU-101

4. Changes from Deliverable 1

The decisions made after Deliverable 1 require adjusting five user stories, creating two new ones and modifying two rules. Each new RF and HU contains a single action, as the professor requested.

RF / HU	Required change	Rule
RF-55 / HU-55	Rewrite the reveal as a step-by-step flow: tap for pinyin, tap for meaning, and a Sentences button for the examples. The grading buttons appear once the meaning is visible, and the space bar advances one step.	Formerly RN-35
New RF / HU	Add one of the user's own decks to another of their lists.	RN-29.1
New RF / HU	Remove a deck from a list without deleting it, as long as it remains in another list.	RN-29.1

HU-46	Deleting a list deletes only the decks that are not in another list. The confirmation states how many decks will be deleted and how many will be kept.	RN-29.2
HU-50	Clarify that deleting a deck removes it from every list where it appears.	RN-29.3
RF-112 / HU-112	Split into "Role assignment" and "Role removal". Criterion 3 becomes "when the teacher role is removed".	RN-18.1, RN-18.3
HU-92	Specify that the assigned deck appears in the course list each enrolled student sees.	RN-33.1
RN-04	The session stores its queue so it can be resumed, instead of never storing it.	RN-04.2
RN-10	The active day is stored when the session completes and is not recalculated when the time zone changes.	RN-10.3

4.1 Rules retired or moved

Four statements left the business rules because they describe security policy, interface behavior or technical limits. Their numbers are not reused.

Former rule	Content	Moves to	Reason
RN-20	Five failed sign-in attempts in 15 minutes lock the account for 15 minutes	RNF-22	Security policy
RN-21	A reset link is single-use, expires after one hour, and using it signs out other devices	RNF-23	Security policy

RN-35	Step-by-step reveal of a flashcard	RF-55, rewritten	Interface behavior
Part of RN-52	Import files must be CSV or JSON of up to 10 MB	A new RNF on import limits	Technical limit

4.2 Rules added in this version

Rule	Why it was added
RN-53	Ley 1581 de 2012 requires informed consent; if the policy changes, earlier consent no longer covers the new terms.
RN-54	Ley 1581 de 2012 (art. 8) gives data subjects the right to know, update and correct their data, to access it free of charge and to request its deletion.
RN-55	HU-112 makes a course inactive when its teacher loses the role, but no rule said what students see afterwards.

5. References

- Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587. https://cancilleria.gov.co/sites/default/files/Normograma/docs/ley_1581_2012.htm
- HanziApp team. (2026). *Deliverable 1: Requirements and user stories*. Ingeniería de Software II, Universidad Nacional de Colombia.
- Mandarin Bean. (2026, April 7). *New HSK vocabulary (2026)*. Retrieved October 9, 2026, from https://mandarinbean.com/new-hsk-vocabulary/
- Open Spaced Repetition. (n.d.). *The algorithm*. Awesome FSRS wiki, GitHub. Retrieved October 9, 2026, from https://github.com/open-spaced-repetition/awesome-fsrs/wiki/The-Algorithm
- Wiegers, K., & Beatty, J. (2013). *Software requirements* (3rd ed.). Microsoft Press.

The HSK 3.0 vocabulary loaded in HanziApp comes from the Mandarin Bean lists, which follow the 2025 official syllabus. Their HSK 1 list has 300 words, the same number loaded in the database, and their English glosses are the official meaning anchor for content

generation. The business-rule taxonomy in section 3 follows Wiegers and Beatty, and the definition of stability in the glossary follows the FSRS documentation.

CRC Cards

These are HanziApp's 22 main classes, one CRC card each. A class is considered main when it has behavior of its own and takes part in several user stories, or when it is the parent of classes that do. Classes that only store a record (PolicyAcceptance, AuditLog, ImportError, among others) appear as collaborators instead of having a card.

Every card follows the standard CRC format: a header with the class name, its type, its superclass and its subclasses; a one-line description; and a two-column body with the responsibilities on the left and the collaborators on the right. Responsibilities state what the class knows or does; collaborators are the classes it needs to ask for something in order to fulfill them.

There are three kinds of class. **Entity** classes hold domain information, such as User or Deck. **Abstract** classes hold what several entities share and are never created directly, such as ReviewState. **Control** classes encapsulate behavior, such as FSRSScheduler: this is how RNF-53 is met, since it requires the scheduling algorithms to be decoupled modules. The superclasses and subclasses match the class diagram.

Class names are in English. The traceability table at the end maps each one to its Spanish name in the domain model.

1. Accounts and access

User

Type: entity · **Superclass:** — · **Subclasses:** —

A person with an account: their identity, access methods, roles, settings and account status.

Responsibilities	Collaborators
Know its name, email and account status: pending, verified or suspended	—

Know its roles and answer whether it can perform an action	Role
Know its access methods and link Google when the email matches	AccessMethod
Record acceptance of the current data policy	PolicyAcceptance, DataPolicy
Ask for acceptance of a new data-policy version at the next sign-in, before anything else	DataPolicy, PolicyAcceptance
Let the user consult and correct their personal data	LearnerProfile
Know its settings: study language, time zone, daily goal, new-word limit and reminders	—
Convert an instant into its local day according to its time zone	—
Store its questionnaire answers and recommend the starting deck based on its declared level	LearnerProfile, DeckList
Request the teacher role	TeacherRequest
Suspend and reactivate itself, leaving a record	AuditLog
Delete itself along with its personal data, lists, decks, progress and enrollments	DeckList, Deck, RecognitionState, WritingState, Enrollment

2. Content

Word

Type: entity · **Superclass:** — · **Subclasses:** —

An HSK 3.0 vocabulary item with a specific reading, such as 你好 (nǐ hǎo); the center of all content.

Responsibilities	Collaborators
Know its hanzi, pinyin, HSK level, audio and whether it is active	—
Know its senses and examples, in order	Sense, Example
Know the characters it is made of, in order	Character, WordComposition
Provide its content in a language only if it is approved in that language	LanguageContent
Express its pinyin in the search forms: with tone marks, with numbers and without tones	—
Deactivate and reactivate itself without affecting users' progress	AuditLog
Receive error reports about its content	ErrorReport

TranslatableElement

Type: abstract · **Superclass:** — · **Subclasses:** Sense, Example

What senses and example sentences share: they are the same in every language and have one translation per language.

Responsibilities	Collaborators
Provide its translation in a given language	Translation
Ensure it has at most one translation per language content	Translation, LanguageContent
Keep its Chinese text and order the same in every language	—

Dictionary

Type: control · **Superclass:** — · **Subclasses:** —

The search service over the vocabulary; it holds no data of its own.

Responsibilities	Collaborators
Search by hanzi: exact match first, then words containing the character, ordered by level	Word
Search by pinyin with or without tones, with marks or numbers, ignoring spaces and case, and treating lü, lv and lu: as equivalent	Word
Search by meaning in the study language, ignoring accents and case, using the first sense to rank results	LanguageContent, User
Filter results by one or several HSK levels	Word
Exclude words that are deactivated or have no approved content in the user's language	Word, LanguageContent

List the words that contain a character	Character, WordComposition

LanguageContent

Type: entity · **Superclass:** — · **Subclasses:** —

A word's content in one language, Spanish or English, with its editorial status from draft to approved.

Responsibilities	Collaborators
Know its language, its status (pending, draft, approved or rejected) and its origin (AI, manual or imported)	—
Group the translations of the senses and examples in its language	Translation, TranslatableElement
Request its validation when created and every time it is edited	ContentValidator, ValidationResult
Get approved only if it has no failures, replacing the previously approved content in the same language	ValidationResult, AuditLog
Get rejected with a reason, leaving the word pending in its language	AuditLog
Keep the generated version so it can be compared with the corrected one	—

ContentGenerator

Type: control · **Superclass:** — · **Subclasses:** —

The service that turns words into draft content with the language model.

Responsibilities	Collaborators
Build batches of up to 100 words in one language, skipping words that already have approved content in it	Word, LanguageContent, GenerationBatch
Send the batch to the language model, using the official gloss as the meaning anchor	Language model (external)
Create a draft per word with senses, examples and translations	LanguageContent
Report the batch's progress and flag the words that failed so they can be retried	GenerationBatch

ContentValidator

Type: control · **Superclass:** — · **Subclasses:** —

The automatic quality check every draft goes through before an administrator can approve it.

Responsibilities	Collaborators
Check that the pinyin matches the hanzi	Word
Check that every example contains the word's hanzi	Example
Check that every translation is entirely in its language	Translation
Detect identical or nearly identical examples	Example

Record each failure with its reason	ValidationResult

VocabularyImporter

Type: control · **Superclass:** — · **Subclasses:** —

The service that loads vocabulary in bulk from a file.

Responsibilities	Collaborators
Accept only CSV or JSON files within the import limits	Import
Validate each row (required fields, level from 1 to 6) and record its errors by row number	ImportError
Create new words with pending content in both languages, without duplicating hanzi and pinyin	Word, LanguageContent
Return the summary: words created, duplicated and with errors	Import

3. Lists and decks

DeckList

Type: entity · **Superclass:** — · **Subclasses:** —

A group of decks: one of the six system HSK lists, or a list a user creates to organize their own decks.

Responsibilities	Collaborators
Know its name, its type (system or personal), its owner and, if it is a system list, its HSK level	User

Know its decks in order and change that order	ListDeck, Deck
Add and remove decks	ListDeck, Deck
Ensure its deck names are not repeated	Deck
When deleted, delete only the decks that belong to no other list	Deck
Reject changes from students and teachers when it is a system list	User

Deck

Type: entity · **Superclass:** — · **Subclasses:** —

A group of words studied together; it can belong to the system, to a user or to a course.

Responsibilities	Collaborators
Know its name, its type (system, personal or course) and its owner	User, Course
Know its words in order, and add or remove words	DeckWord, Word
If it is a system deck, accept only words of its level that are not in another deck of the same list	DeckList, Word
Compute its summary: total words, new words, reviews due today and percentage learned	RecognitionState, ProgressCalculator
Copy itself as an independent personal deck into one of the user's lists	DeckList

Tell whether a session can be started, that is, whether it has words	—
When deleted, remove itself from all its lists and discard its in-progress sessions	ListDeck, StudySession

4. Review and sessions

StudySession

Type: entity · **Superclass:** — · **Subclasses:** —

One study session on a deck, or on all due reviews, which can be paused and resumed on any device.

Responsibilities	Collaborators
Know its scope (scheduled, free review or all due reviews), its practice type, its writing mode and its status	—
Ensure a single in-progress session per user, deck and practice type	User, Deck
Get its queue when it starts and again when it is resumed	SessionQueueBuilder, SessionItem
Present the next card and, in writing, each character of the word in order, remembering the position so it can be resumed on any device	SessionItem, Word, WritingExercise
Record each answer, a grade or a written character, and request the state update except in free review	PracticeLog, RecognitionState, WritingState

Reinsert a card graded Hard after at least 3 other cards, and drop it after the third time	SessionItem
Undo the last grade	PracticeLog, RecognitionState
Discard itself while keeping the grades already recorded	PracticeLog
When completed, record the active day and return its summary	ActiveDay, ProgressCalculator

SessionQueueBuilder

Type: control · **Superclass:** — · **Subclasses:** —

The service that decides which words a session presents and in what order, so StudySession only has to run the session.

Responsibilities	Collaborators
Build a scheduled queue: the deck's due words first, then new words up to the daily limit	Deck, RecognitionState, WritingState
Apply the daily new-word limit across all decks, without counting free review	ProgressCalculator, User
Never present a word already studied as new in another deck	RecognitionState
Build a free review: every word of the deck in random order	Deck

Build the all-due-reviews queue: due words from all the user's decks, each once and with no new words	DeckList, Deck, RecognitionState
Recalculate a scheduled queue when the session is resumed, dropping words that are no longer due	SessionItem, RecognitionState

WritingExercise

Type: control · **Superclass:** — · **Subclasses:** —

One attempt at writing a character, stroke by stroke.

Responsibilities	Collaborators
Present a character in easy mode (faint template and next-stroke hint) or in hard mode (only pinyin and meaning)	Character, Word
Validate each stroke by shape, direction and order, within a tolerance margin	Character, Stroke data (external)
Count errors and, in hard mode, show a hint after three consecutive errors on the same stroke	—
Record "I don't know" and animate the whole character	Character
Return the result to the session: errors, hints, mode and whether it was "I don't know"	StudySession

ReviewState

Type: abstract · **Superclass:** — · **Subclasses:** RecognitionState, WritingState

What every review state shares: whose it is, when the item was last reviewed and when it is due again.

Responsibilities	Collaborators
Belong to exactly one user	User
Know the last and next review dates	—
Tell whether its review is due on a given local day	User
Leave to each subclass the decision of when its item counts as learned	—

RecognitionState

Type: entity · **Superclass:** ReviewState · **Subclasses:** —

How well a user recognizes one word, scheduled with FSRS; there is one per user and word, whatever the deck.

Responsibilities	Collaborators
Know the stability, difficulty and stage of a word for a user	Word
Tell whether the word is new, in learning or learned (stability of 21 days or more)	—
Update itself with a grade, delegating the calculation	FSRSScheduler
Restore its previous state when a grade is undone	PracticeLog

FSRSScheduler

Type: control · **Superclass:** — · **Subclasses:** —

The FSRS algorithm, isolated so it can be tuned or replaced without touching the rest of the system.

Responsibilities	Collaborators
Compute the new stability, difficulty and next review date from the current state and the grade	RecognitionState
Translate Easy into an FSRS success and Hard into an FSRS lapse	—
Read its parameters from configuration, without depending on the rest of the system	—

WritingState

Type: entity · **Superclass:** ReviewState · **Subclasses:** —

How well a user can write one character from memory; there is one per user and character, whatever the word.

Responsibilities	Collaborators
Know the interval and whether the character has been practiced in hard mode	Character
Tell whether the character is learned: interval of 21 days or more, practiced in hard mode	—
Update itself with the result of an exercise, delegating the calculation	WritingScheduler

WritingScheduler

Type: control · **Superclass:** — · **Subclasses:** —

HanziApp's own motor-memory algorithm for handwriting reviews.

Responsibilities	Collaborators
Compute the next interval from the errors, hints used and mode	WritingState
Give a shorter interval in easy mode than in hard mode for the same performance	—
Reset the interval when the user marks "I don't know"	—
Read its parameters from configuration, without depending on the rest of the system	—

PracticeLog

Type: abstract · **Superclass:** — · **Subclasses:** ReviewLog, WritingLog

The record of a single answer in a session: the history behind undo, the new-word limit and the statistics.

Responsibilities	Collaborators
Know the session and the moment the answer was given	StudySession
Know whether the answer affects scheduling, which is false in free review	—
For a grade, keep the previous state so it can be undone	RecognitionState

Provide the history for the new-word limit and the statistics	ProgressCalculator

5. Progress and motivation

ProgressCalculator

Type: control · **Superclass:** — · **Subclasses:** —

The service that turns review states and active days into the numbers the student sees; it stores nothing itself.

Responsibilities	Collaborators
Compute the current and longest streak from the active days	ActiveDay, User
Tell whether the streak is at risk today, for the 8:00 p.m. reminder	ActiveDay, User
Count distinct learned words, excluding deactivated ones	RecognitionState, Word
Count distinct learned characters	WritingState
Compute the progress percentage for each HSK level	RecognitionState, Word
Compute today's due reviews, separately for words and characters	RecognitionState, WritingState, User
Count the new words studied today in scheduled sessions, to enforce the daily limit	PracticeLog, User

Compute the minutes studied today against the daily goal	StudySession, User
Award a badge when a milestone is reached, only once per user	Badge, EarnedBadge

6. Courses

Course

Type: entity · **Superclass:** — · **Subclasses:** —

A group a teacher creates to assign decks to their students and follow their progress.

Responsibilities	Collaborators
Know its name, its teacher, its invitation code and whether it is active	User
Generate a unique 8-character code and regenerate it, invalidating the previous one	—
Enroll students with a valid code and remove them when they leave	Enrollment
Know its decks and assign to enrolled students the ones that have words	Deck
Present itself to each enrolled student as a read-only list containing the assigned decks	Deck, Enrollment
Provide each enrolled student's progress and the group averages	Enrollment, ProgressCalculator

Become inactive if its teacher loses the role	User
While inactive, reject new enrollments and edits, but keep its list for students already enrolled	Enrollment

Enrollment

Type: entity · **Superclass:** — · **Subclasses:** —

A student's membership in a course, including their consent to share progress with the teacher.

Responsibilities	Collaborators
Know the student, the course, the date and the consent to share progress	User, Course
Require the student to agree to share their progress before it is created	—
Expose to the teacher only the streak, active days, learned words and progress on assigned decks, never the email or the questionnaire	ProgressCalculator
When it ends, remove the course decks from the student's lists and discard their in-progress sessions	Course, StudySession

7. Traceability

Each class answers to a group of functional requirements and System Domain business rules. The Spanish name column matches the domain model.

Class	Spanish name	Type	RF	RN

User	Usuario	Entity	RF-01 to RF-23, RF-26 to RF-30, RF-82, RF-83, RF-87, RF-111 to RF-115	RN-15.1 to RN-15.4, RN-16.1 to RN-16.3, RN-17.1, RN-17.2, RN-18.1 to RN-18.3, RN-19.1, RN-19.2, RN-22.1 to RN-22.4, RN-23.1, RN-23.2, RN-53, RN-54
Word	Palabra	Entity	RF-31, RF-36 to RF-39, RF-42, RF-96 to RF-98, RF-107	RN-26.1, RN-26.2, RN-27.1, RN-27.2, RN-28
TranslatableElement	ElementoTraducible	Abstract	RF-36	RN-24
Dictionary	Diccionario	Control	RF-32 to RF-35, RF-40	RN-15.2, RN-15.3, RN-27.1
LanguageContent	ContenidoIdioma	Entity	RF-99, RF-104 to RF-106	RN-14, RN-24, RN-25.1 to RN-25.3
ContentGenerator	GeneradorContenido	Control	RF-102	RN-14, RN-51.1, RN-51.2
ContentValidator	ValidadorContenido	Control	RF-103	RN-25.3
VocabularyImporter	ImportadorVocabulario	Control	RF-100, RF-101	RN-26.1, RN-52.1, RN-52.2

DeckList	Lista	Entity	RF-43 to RF-46, RF-49, RF-110, new RFs for adding and removing a deck from a list	RN-03, RN-13, RN-29.1 to RN-29.3, RN-30.1, RN-33.1
Deck	Mazo	Entity	RF-41, RF-47, RF-48, RF-50 to RF-53, RF-108, RF-109	RN-03, RN-29.3, RN-30.2, RN-31.1 to RN-31.3, RN-32, RN-34
StudySession	SesionEstudio	Entity	RF-54, RF-55, RF-58, RF-60, RF-67, RF-71 to RF-74	RN-04.2, RN-12.1, RN-12.2, RN-36.1, RN-36.2, RN-42.1, RN-42.2, RN-43
SessionQueueBuilder	ConstructorColaSesion	Control	RF-59, RF-68 to RF-70, RF-72	RN-04.1, RN-04.3, RN-04.4, RN-12.3, RN-37.1, RN-37.3, RN-40, RN-41
WritingExercise	EjercicioEscritura	Control	RF-61 to RF-65	RN-39
ReviewState	EstadoRepaso	Abstract	RF-57, RF-66	RN-01, RN-02

RecognitionState	EstadoReconocimiento	Entity	RF-56, RF-57	RN-01, RN-05.1, RN-05.2, RN-06
FSRSScheduler	ProgramadorFSRS	Control	RF-57	RN-05.1, RN-05.2
WritingState	EstadoEscritura	Entity	RF-66	RN-02, RN-07, RN-38.3
WritingScheduler	ProgramadorEscritura	Control	RF-66	RN-38.1 to RN-38.3
PracticeLog	RegistroPractica	Abstract	RF-60, RF-71	RN-40, RN-42.1
ProgressCalculator	CalculadoraProgreso	Control	RF-59, RF-75 to RF-81, RF-84	RN-06 to RN-09, RN-10.1 to RN-10.3, RN-11, RN-37.1, RN-44, RN-45
Course	Curso	Entity	RF-88, RF-89, RF-91 to RF-94	RN-33.1, RN-33.2, RN-46.2, RN-47.1, RN-55
Enrollment	Inscripcion	Entity	RF-90, RF-95	RN-33.3, RN-43, RN-46.1, RN-47.1 to RN-47.3

Three requirements have no main class of their own: the landing page (RF-24 and RF-25) is presentation only, and usage metrics (RF-116) are computed with aggregate queries over sessions and users.

Collaborator classes without a card

Class	Spanish name
Role	Rol
AccessMethod	MetodoAcceso
DataPolicy	PoliticaDatos
PolicyAcceptance	AceptacionPolitica
LearnerProfile	PerfilAprendizaje
TeacherRequest	SolicitudProfesor
AuditLog	RegistroAuditoria
Character	Caracter
WordComposition	ComposicionPalabra
Sense (subclass of TranslatableElement)	Acepcion
Example (subclass of TranslatableElement)	Ejemplo
Translation	Traduccion

ValidationResult	ResultadoValidacion
ErrorReport	ReporteError
GenerationBatch	LoteGeneracion
Import	Importacion
ImportError	ErrorImportacion
ListDeck	ListaMazo
DeckWord	MazoPalabra
SessionItem	ItemSesion
ReviewLog (subclass of PracticeLog)	RegistroRepaso
WritingLog (subclass of PracticeLog)	RegistroEscritura
ActiveDay	DiaActivo
Badge	Insignia
EarnedBadge	LogroObtenido

Class Diagram

HanziApp's class diagram has 52 classes, one of them an interface, and 22 enumerations, organized into eight packages. It builds on the domain model and the 22 CRC cards: each card's responsibilities become methods, and its collaborators become associations or dependencies. Class names match the English CRC cards.

An overview without detail comes first, followed by one diagram per package with attributes, methods, multiplicities, navigability and constraints. The Learning package is split into two diagrams, review states and study sessions, so neither becomes unreadable. Enumerations get a diagram of their own.

How to read the diagram

UML notation

The notation follows the UML 2.5 conventions described on uml-diagrams.org.

Element	Meaning
`-` and `+`	Private attribute and public method
`/name`	Derived attribute: computed from other data, never stored (for example `/currentStreak`)
Name in italics	Abstract class: no objects are created from it, only from its subclasses
`{static}` (underlined)	Class-level method, called without an instance
`{...}` next to an attribute	Constraint on its values, such as `{1..6}` or `{max 100}`
Note with `{...}`	Constraint involving several objects, such as `{unique (user, word)}`

Solid line with multiplicities	Association (`1`, `0..1`, `0..*`, `1..*`)
Open arrowhead on an association	Navigability: the class at the tail knows the class at the head, not the other way around
Name with ▸ or ◂	Association name and its reading direction, as in *StudySession* ▸ *covers Deck*
Filled diamond	Composition: the part cannot exist without the whole (a Sense without its Word)
Hollow diamond	Aggregation: the whole groups parts that also belong elsewhere (LanguageContent groups Translations)
Solid line with hollow triangle	Generalization: the subclass is a specific case of the superclass
Dashed line with hollow triangle	Realization: the class implements the interface
Dashed arrow with `«use»`	Usage dependency: the class calls the other to fulfill a responsibility
Dashed arrow with `«create»`	Creation dependency: the class creates instances of the other
`«interface»`, `«control»`, `«datatype»`, `«enumeration»`	Interface, control class, data type without identity, and closed set of values

Types are generic (String, Integer, Boolean, Float, Date, DateTime) so the diagram does not depend on a programming language. Identifiers are omitted: they appear as primary keys in the relational model.

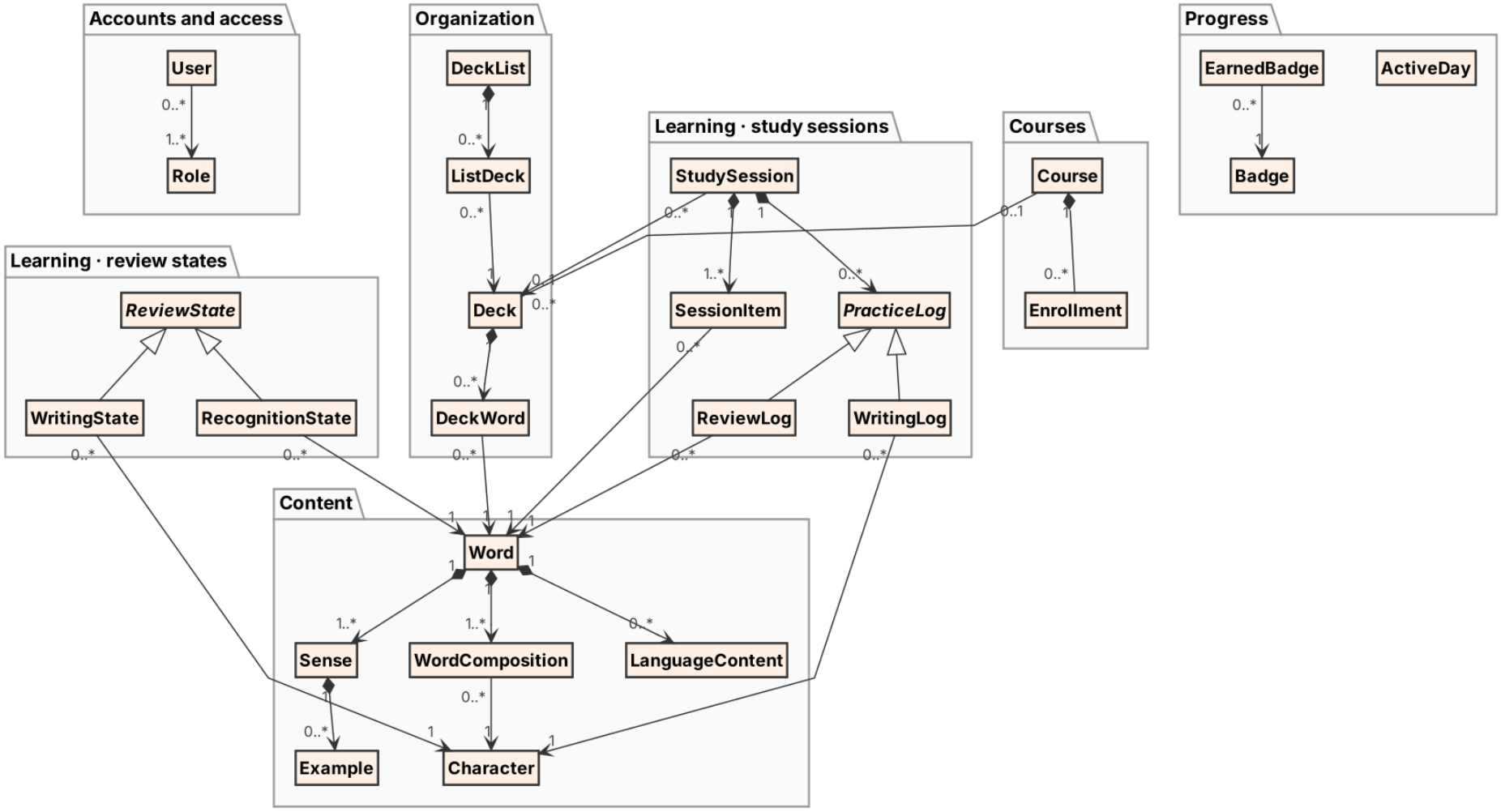
Design decisions

1. **Three inheritance hierarchies.**
    - RecognitionState and WritingState inherit from ReviewState, because both store the last and next review and answer whether they are due on a given day.
    - Sense and Example inherit from TranslatableElement, so every Translation belongs to exactly one element, whether a sense or an example (RN-24).
    - ReviewLog and WritingLog inherit from PracticeLog, which stores the timestamp and the session.
2. **The algorithms sit behind an interface.** FSRSScheduler and WritingScheduler realize the `ReviewScheduler` interface and know nothing about the database: they receive a state, return the next one and read their parameters from configuration. Either algorithm can be replaced without touching the rest of the system (RNF-53, RNF-54).
3. **Navigability points toward shared content.** Learning and organization classes know the Word or Character they refer to, but a Word does not keep a list of every user's states, decks or sessions. Associations with User stay bidirectional because both sides need them.
4. **What can be computed is not stored.** The streak, retrievability, deck size and number of students are derived attributes (`/`), and ProgressCalculator computes them (RN-06 to RN-11).
5. **Statuses and types are enumerations.** This avoids free-text values such as the current `pos_raw` field or unvalidated HSK levels.
6. **Relationships with their own data are intermediate classes.** ListDeck, DeckWord, WordComposition and Enrollment store the order, position or consent of the relationship. They are drawn as regular classes rather than association classes because they map directly to tables in the relational model.
7. **Key business rules appear as constraints.** Notes on the diagrams state the uniqueness rules that shape the model: one state per user and word (RN-01) or character (RN-02), one in-progress session per deck and practice type (RN-12.2), and one approved content per word and language (RN-25.2).

1. Overview

The overview shows the 25 core classes and how they connect, without attributes or methods. Word is the hub of the model: almost every learning and organization class points to it. User is associated with a class in every package; to keep the overview readable, those links appear only in each package diagram.

HanziApp · Overview (core classes, without attributes or methods)

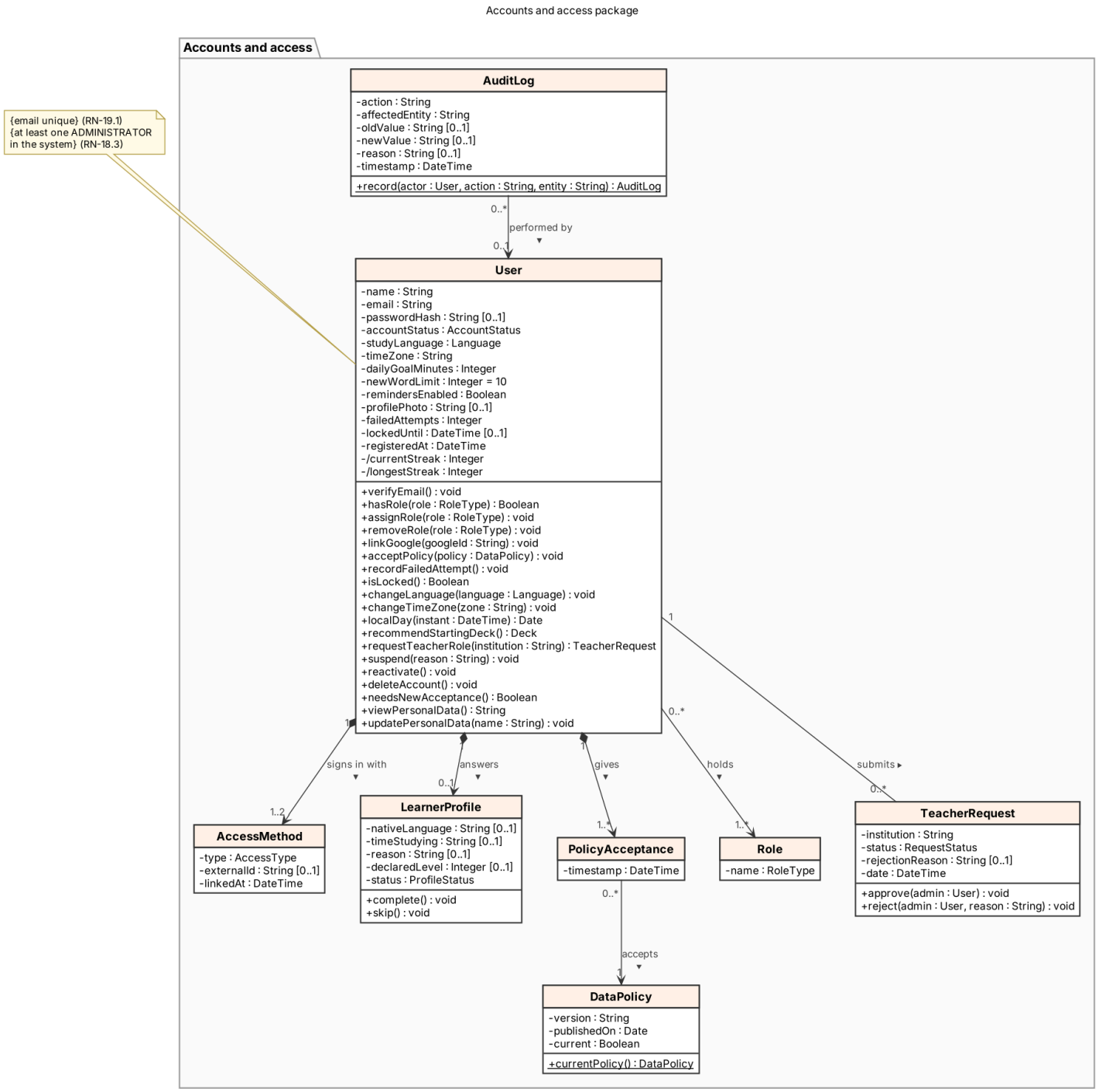


User is associated with classes in every package; those links are shown in each package diagram.

Each package depends only on packages earlier in this chain, never the other way around: Accounts and access → Content → Content management → Organization → Courses → Learning · review states → Learning · study sessions → Progress. That is why each cross-package relationship is drawn in the diagram of the dependent package.

2. Accounts and access package

User holds the account's identity, settings and status; the other classes in the package record how the user signs in, what they accepted and what they requested.



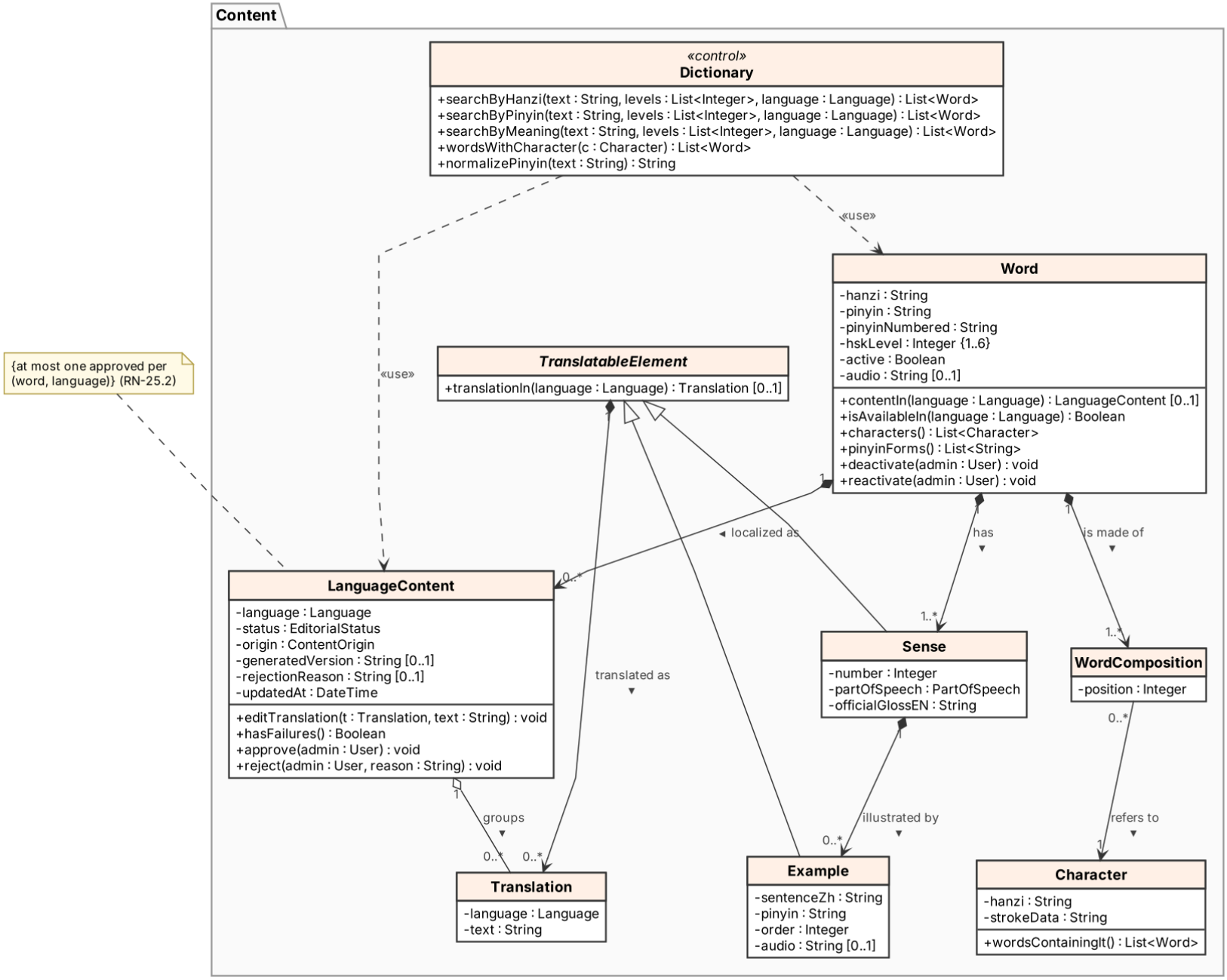
- **Several roles per user:** User ▸ holds Role is `0..*` to `1..*` and navigable toward Role, so a teacher can also be a student (RN-18.1).

- **One or two access methods:** email, Google or both linked (RN-19.2). That is why `passwordHash` is optional `[0..1]`: an account created with Google has no password.
- **Versioned policy:** each PolicyAcceptance points to the exact DataPolicy version the user accepted (RN-17.2). `needsNewAcceptance()` asks again when a new version is published (RN-53), and `viewPersonalData()` and `updatePersonalData()` support the data subject's rights (RN-54).
- **Derived streaks:** `/currentStreak` and `/longestStreak` are computed from the active days, never stored (RN-11, RN-44).
- **Audit:** AuditLog ▸ performed by User is `0..1`, so the history survives when an account is deleted. `AuditLog.record()` is static because administrative actions in every package call it (RN-49).
- **Constraints in the note:** the email is unique (RN-19.1), and there is always at least one administrator (RN-18.3).

3. Content package

Content separates what is the same in every language (the word, its characters, its senses and its Chinese example sentences) from what changes per language (the translations, grouped in LanguageContent).

Content package

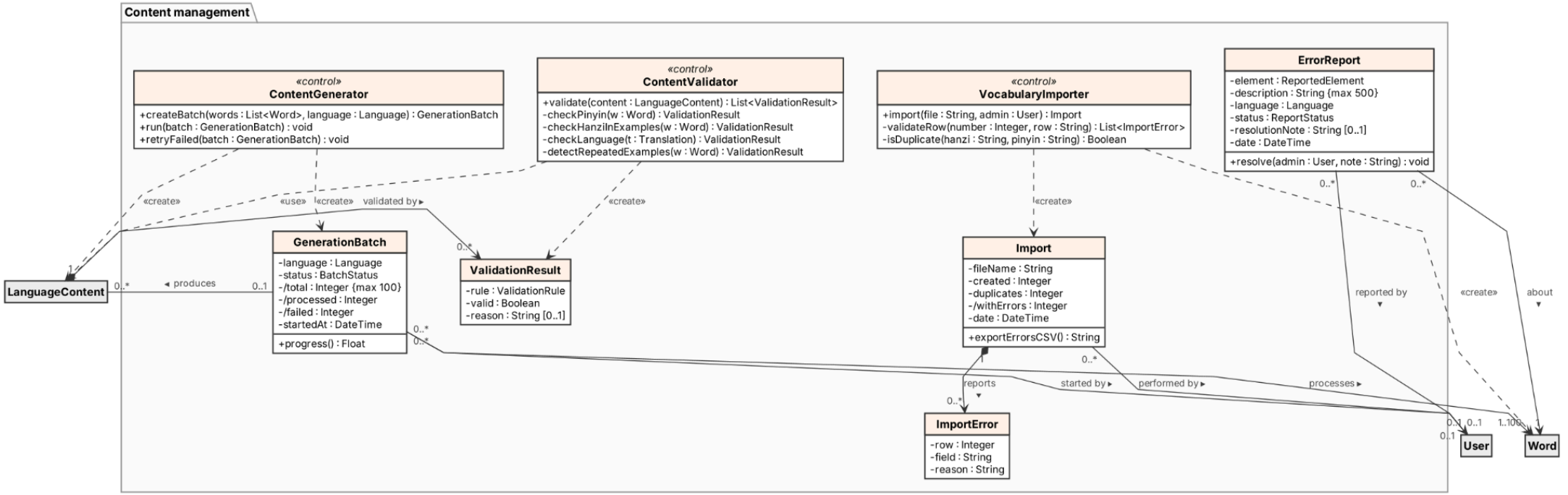


- **TranslatableElement:** Sense and Example inherit from this abstract class, and each one owns its translations by composition, so a Translation always belongs to exactly one element (RN-24).
- **LanguageContent aggregates translations:** the hollow diamond shows that it groups the translations of one language without owning them; approving it publishes all of them at once (RN-25).
- **WordComposition:** an intermediate class between Word and Character that stores each character's position. It is what lets 你好 be practiced in the order 你, 好 (RN-28).
- **PartOfSpeech:** replaces the free-text `pos_raw` field with an enumeration.
- **Dictionary:** a control class that resolves the three searches and pinyin normalization; it holds no state and only has `«use»` dependencies.

4. Content management package

Content management gathers the administrator's tools: generating, validating and importing vocabulary, and handling user reports.

Content management package

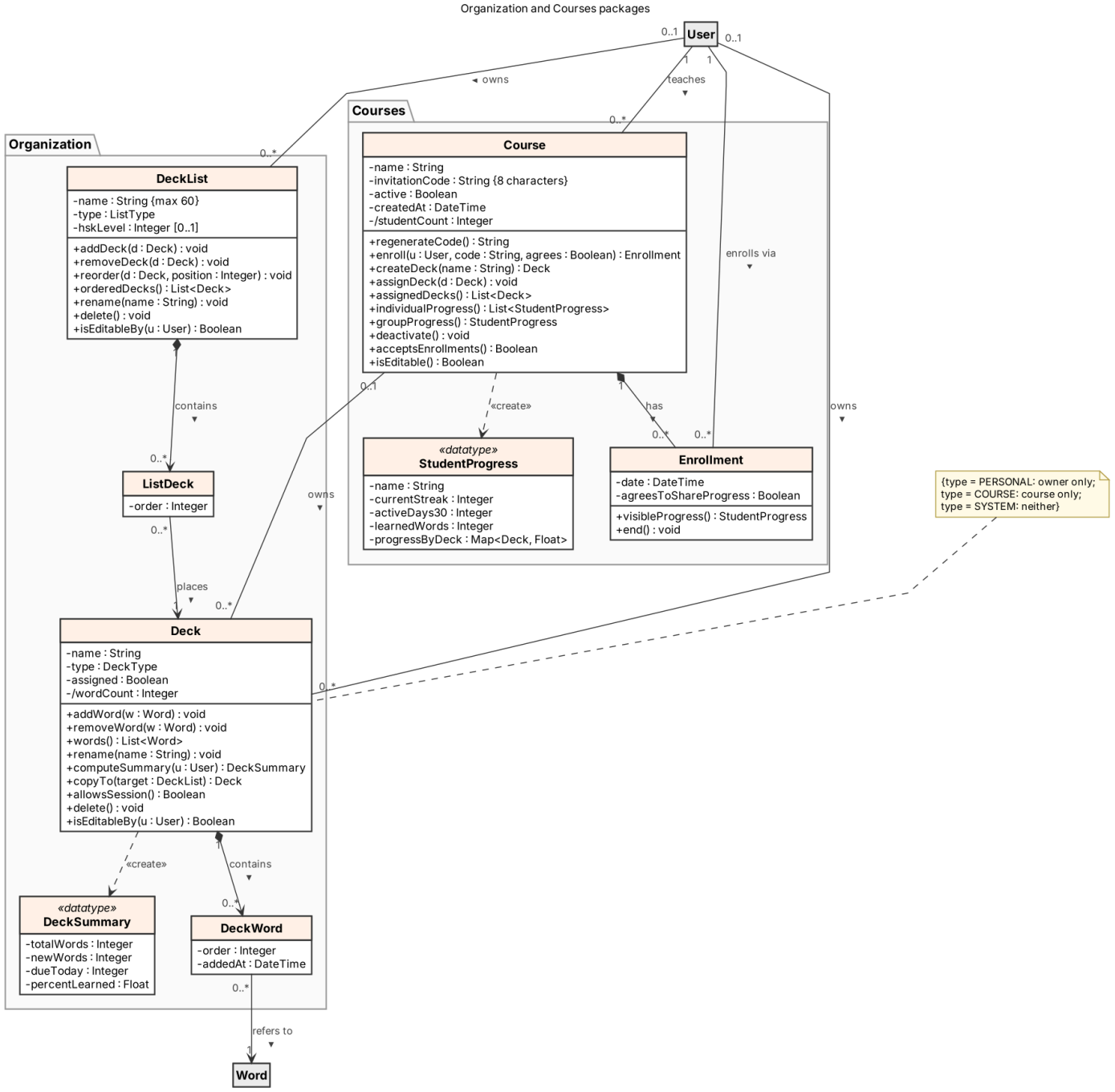


- **ContentGenerator** `«create»`s batches of up to 100 words (`GenerationBatch.total {max 100}`, RN-51.1) and leaves each result as a draft LanguageContent (RN-14).
- **ContentValidator** `«use»`s the content and `«create»`s one ValidationResult per rule (RF-103).
- **VocabularyImporter** validates row by row and records the errors with their row number (RF-101, RN-52.2).
- **Authorship survives deletion:** ErrorReport, GenerationBatch and Import point to User with `0..1`, matching the relational model.

The batch and import counters (`/total`, `/processed`, `/failed` and `/withErrors`) are derived attributes: the database computes them from `generation_batch_words` and `import_errors` instead of storing them.

5. Organization and Courses packages

A DeckList contains decks and a Deck groups words; both relationships are many-to-many and store their order in intermediate classes.

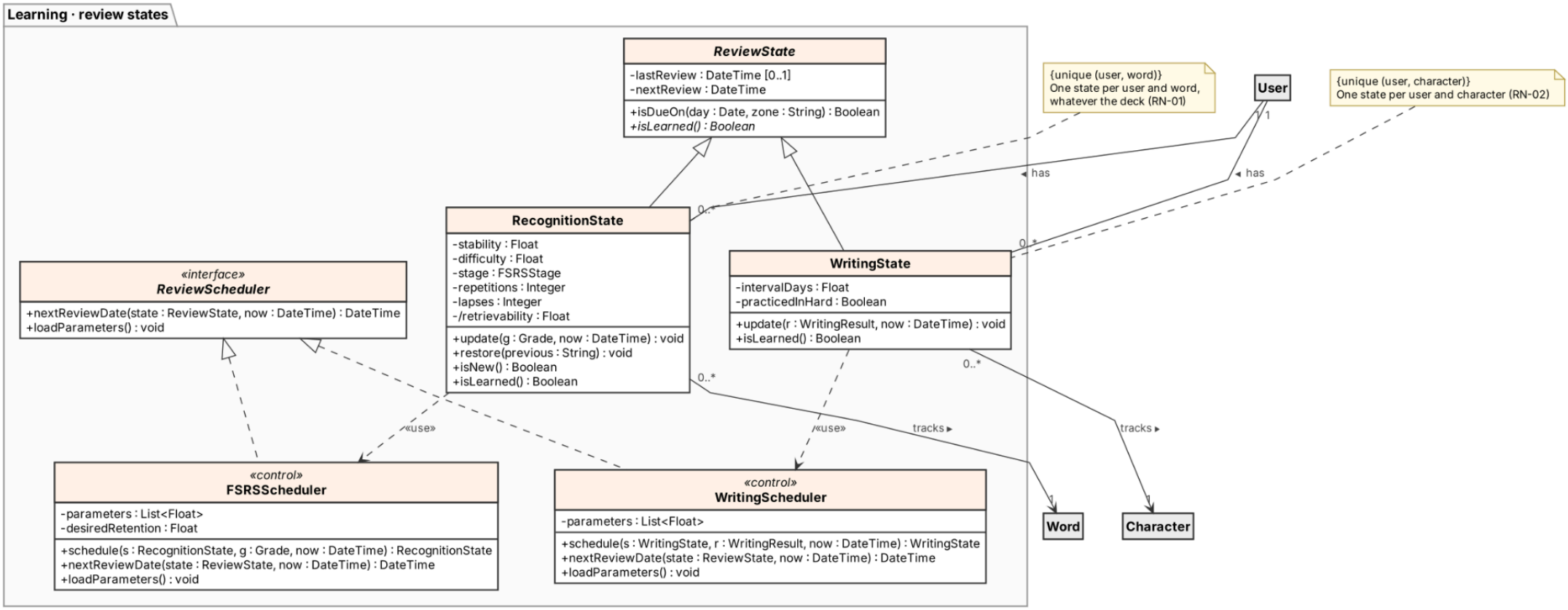

Organization and Courses packages
User
owns
teaches
Courses
Organization
Course
-name : String
-invitationCode : String {8 characters}
-active : Boolean
-createdAt : DateTime
-/studentCount : Integer
+regenerateCode() : String
+enroll(u : User, code : String, agrees : Boolean) : Enrollment
+createDeck(name : String) : Deck
+assignDeck(d : Deck) : void
+assignedDecks() : List<Deck>
+individualProgress() : List<StudentProgress>
+groupProgress() : StudentProgress
+deactivate() : void
+acceptsEnrollments() : Boolean
+isEditable() : Boolean
enrolls via
DeckList
-name : String {max 60}
-type : ListType
-hskLevel : Integer [0..1]
+addDeck(d : Deck) : void
+removeDeck(d : Deck) : void
+reorder(d : Deck, position : Integer) : void
+orderedDecks() : List<Deck>
+rename(name : String) : void
+delete() : void
+isEditableBy(u : User) : Boolean
contains
«create»
has
owns
«datatype»
StudentProgress
-name : String
-currentStreak : Integer
-activeDays30 : Integer
-learnedWords : Integer
-progressByDeck : Map<Deck, Float>
Enrollment
-date : DateTime
-agreesToShareProgress : Boolean
+visibleProgress() : StudentProgress
+end() : void
ListDeck
-order : Integer
places
Deck
-name : String
-type : DeckType
-assigned : Boolean
-/wordCount : Integer
+addWord(w : Word) : void
+removeWord(w : Word) : void
+words() : List<Word>
+rename(name : String) : void
+computeSummary(u : User) : DeckSummary
+copyTo(target : DeckList) : Deck
+allowsSession() : Boolean
+delete() : void
+isEditableBy(u : User) : Boolean
«datatype»
DeckSummary
-totalWords : Integer
-newWords : Integer
-dueToday : Integer
-percentLearned : Float
DeckWord
-order : Integer
-addedAt : DateTime
refers to
Word
{type = PERSONAL: owner only;
type = COURSE: course only;
type = SYSTEM: neither}

- **ListDeck:** lets a deck sit in several lists with a different position in each (RN-29.1). The composition with DeckList means deleting a list deletes its links, not necessarily its decks.
- **DeckWord:** stores the order and the date each word was added, and points to Word without Word knowing its decks. Removing a word deletes the link, never the review state (RN-03).
- **Three deck types, one owner each:** the note states the rule. A personal deck belongs to a User, a course deck to a Course, and a system deck to neither, which is why both associations are `0..1`.
- **Enrollment:** a class between User and Course that records the consent to share progress (RN-46.1). `visibleProgress()` returns only the data the teacher may see (RN-47.1, RN-47.2).
- **Inactive courses:** `acceptsEnrollments()` and `isEditable()` return false while the course is inactive, and its students keep the list (RN-55).
- **Derived counts:** `/wordCount` and `/studentCount` are computed. DeckSummary and StudentProgress are `«datatype»`s that are `«create»`d on request and never stored.

6. Learning · review states package

Review states hold what the user remembers. Each state delegates the scheduling calculation to an algorithm behind the `ReviewScheduler` interface.

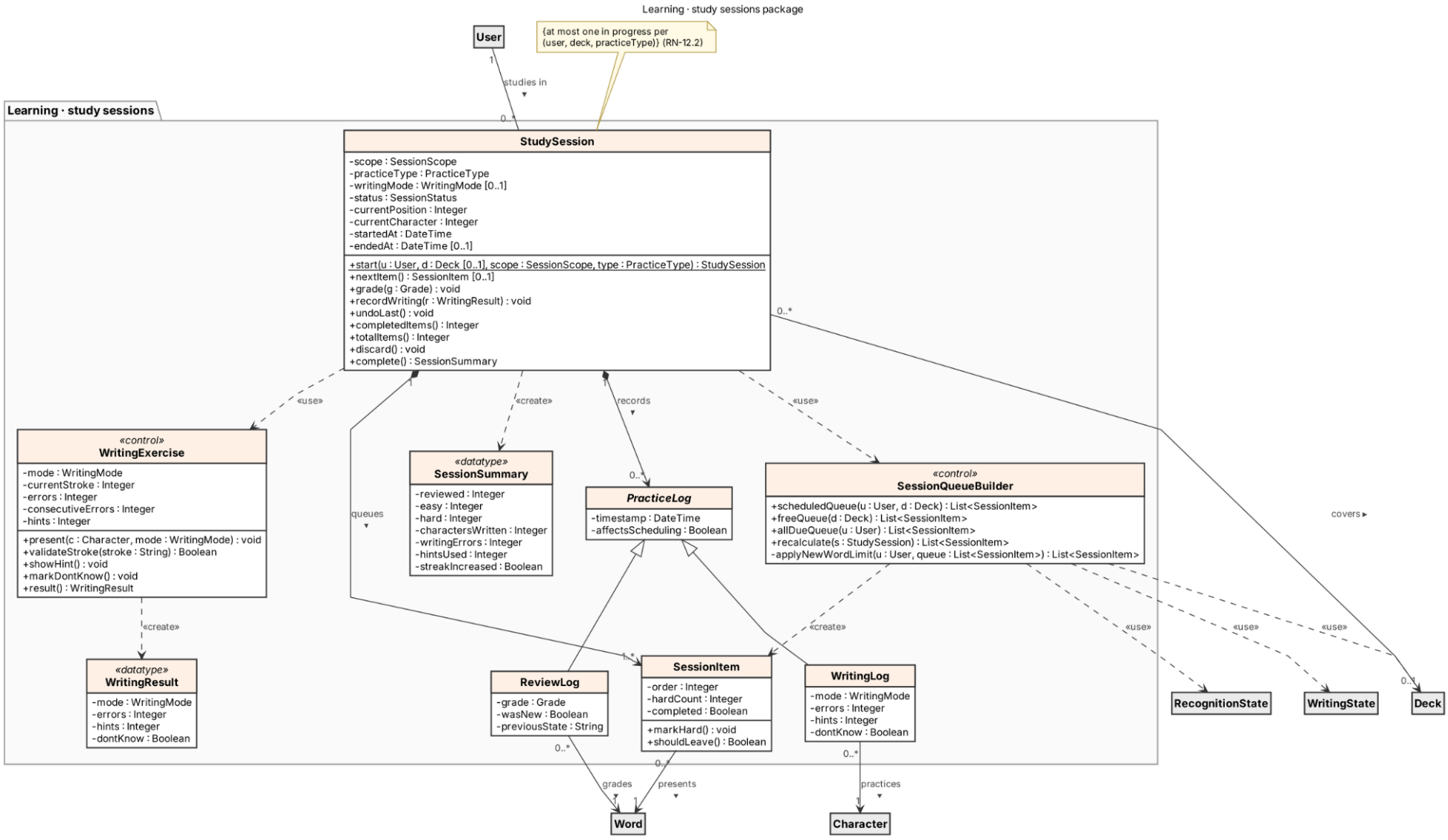
Learning · review states package



- **ReviewState:** RecognitionState (per user and word) and WritingState (per user and character) inherit from this abstract class. Neither is associated with Deck or DeckList; the notes `{unique (user, word)}` and `{unique (user, character)}` make the single global state explicit (RN-01, RN-02).
- **Interface realization:** FSRSScheduler and WritingScheduler realize `ReviewScheduler`, so the session and the states do not depend on a specific algorithm (RNF-53). `loadParameters()` reads the parameters from configuration (RNF-54).
- **Derived retrievability:** `/retrievability` is the FSRS probability of remembering the word right now, computed from stability and elapsed time.

7. Learning · study sessions package

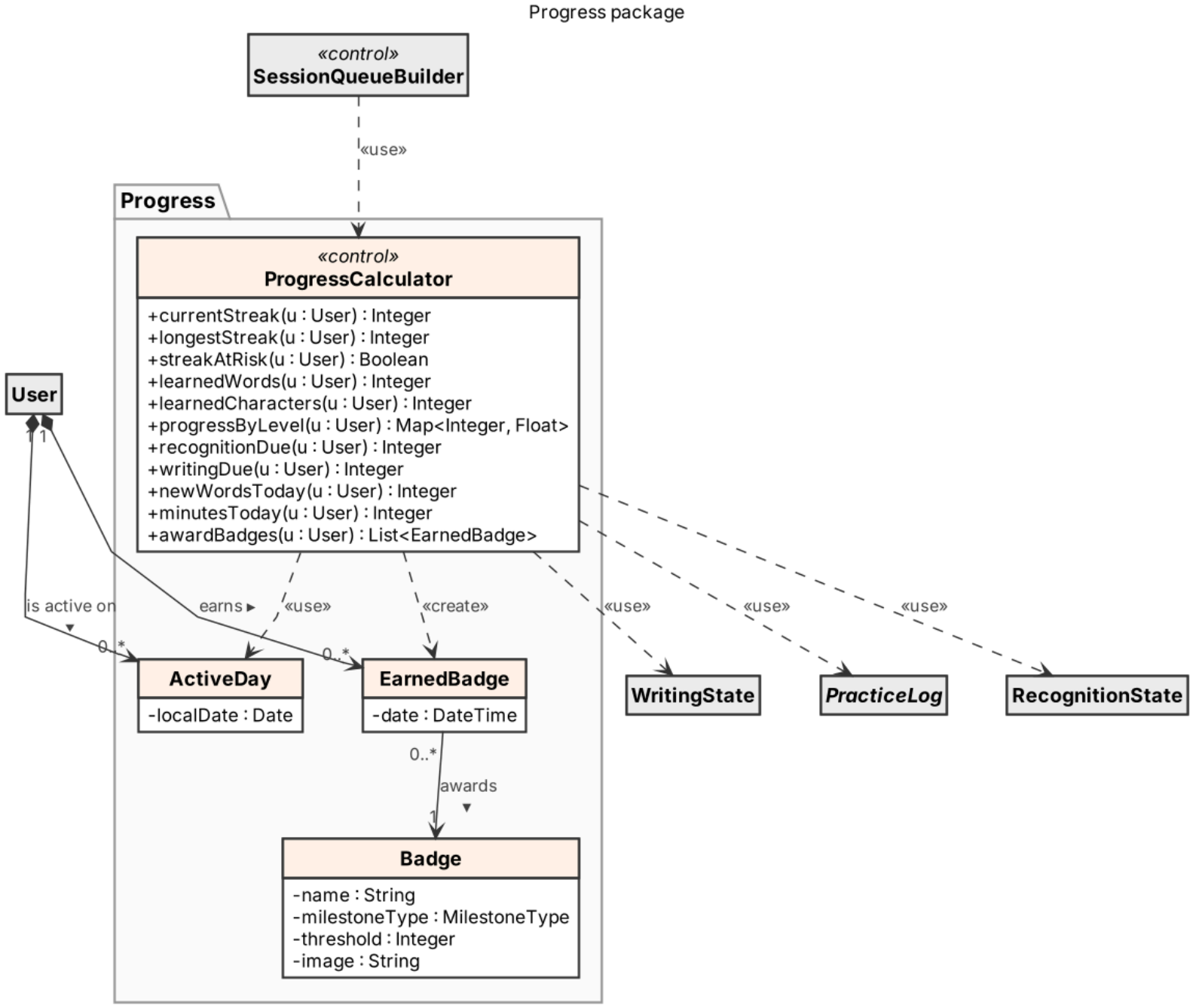
StudySession runs the review; SessionQueueBuilder decides which words it presents and in what order.


Learning · study sessions package
User
{at most one in progress per (user, deck, practiceType)} (RN-12.2)
1
studies in
0..*
Learning · study sessions
StudySession
-scope : SessionScope
-practiceType : PracticeType
-writingMode : WritingMode [0..1]
-status : SessionStatus
-currentPosition : Integer
-currentCharacter : Integer
-startedAt : DateTime
-endedAt : DateTime [0..1]
+start(u : User, d : Deck [0..1], scope : SessionScope, type : PracticeType) : StudySession
+nextItem() : SessionItem [0..1]
+grade(g : Grade) : void
+recordWriting(r : WritingResult) : void
+undoLast() : void
+completedItems() : Integer
+totalItems() : Integer
+discard() : void
+complete() : SessionSummary
0..*
«use»
«create»
1
records
0..*
«use»
«control»
WritingExercise
-mode : WritingMode
-currentStroke : Integer
-errors : Integer
-consecutiveErrors : Integer
-hints : Integer
+present(c : Character, mode : WritingMode) : void
+validateStroke(stroke : String) : Boolean
+showHint() : void
+markDontKnow() : void
+result() : WritingResult
«create»
«datatype»
WritingResult
-mode : WritingMode
-errors : Integer
-hints : Integer
-dontKnow : Boolean
queues
1..*
«datatype»
SessionSummary
-reviewed : Integer
-easy : Integer
-hard : Integer
-charactersWritten : Integer
-writingErrors : Integer
-hintsUsed : Integer
-streakIncreased : Boolean
PracticeLog
-timestamp : DateTime
-affectsScheduling : Boolean
«control»
SessionQueueBuilder
+scheduledQueue(u : User, d : Deck) : List<SessionItem>
+freeQueue(d : Deck) : List<SessionItem>
+allDueQueue(u : User) : List<SessionItem>
+recalculate(s : StudySession) : List<SessionItem>
-applyNewWordLimit(u : User, queue : List<SessionItem>) : List<SessionItem>
«create»
«use»
«use»
«use»
covers ▸
0..1
ReviewLog
-grade : Grade
-wasNew : Boolean
-previousState : String
SessionItem
-order : Integer
-hardCount : Integer
-completed : Boolean
+markHard() : void
+shouldLeave() : Boolean
WritingLog
-mode : WritingMode
-errors : Integer
-hints : Integer
-dontKnow : Boolean
RecognitionState
WritingState
Deck
0..*
grades
1
presents
0..*
0..*
practices
1
Word
Character

- **SessionQueueBuilder** is a control class that `«create»`s every queue: scheduled, free review and all due reviews. It `«use»`s Deck, both review states and ProgressCalculator, applies the new-word limit and recalculates a scheduled queue on resume (RN-04.1, RN-12.3, RN-37.1, RN-40, RN-41).
- **One session in progress:** the note `{at most one in progress per (user, deck, practiceType)}` states RN-12.2.
- **SessionItem:** stores the session queue so it can be resumed, and its `hardCount` implements the three-repetition rule (RN-04.2, RN-36.2).
- **PracticeLog:** its two subclasses store every grade and every written character. `affectsScheduling` is false in free review (RN-40), and `previousState` makes undo possible (RF-60).

8. Progress package

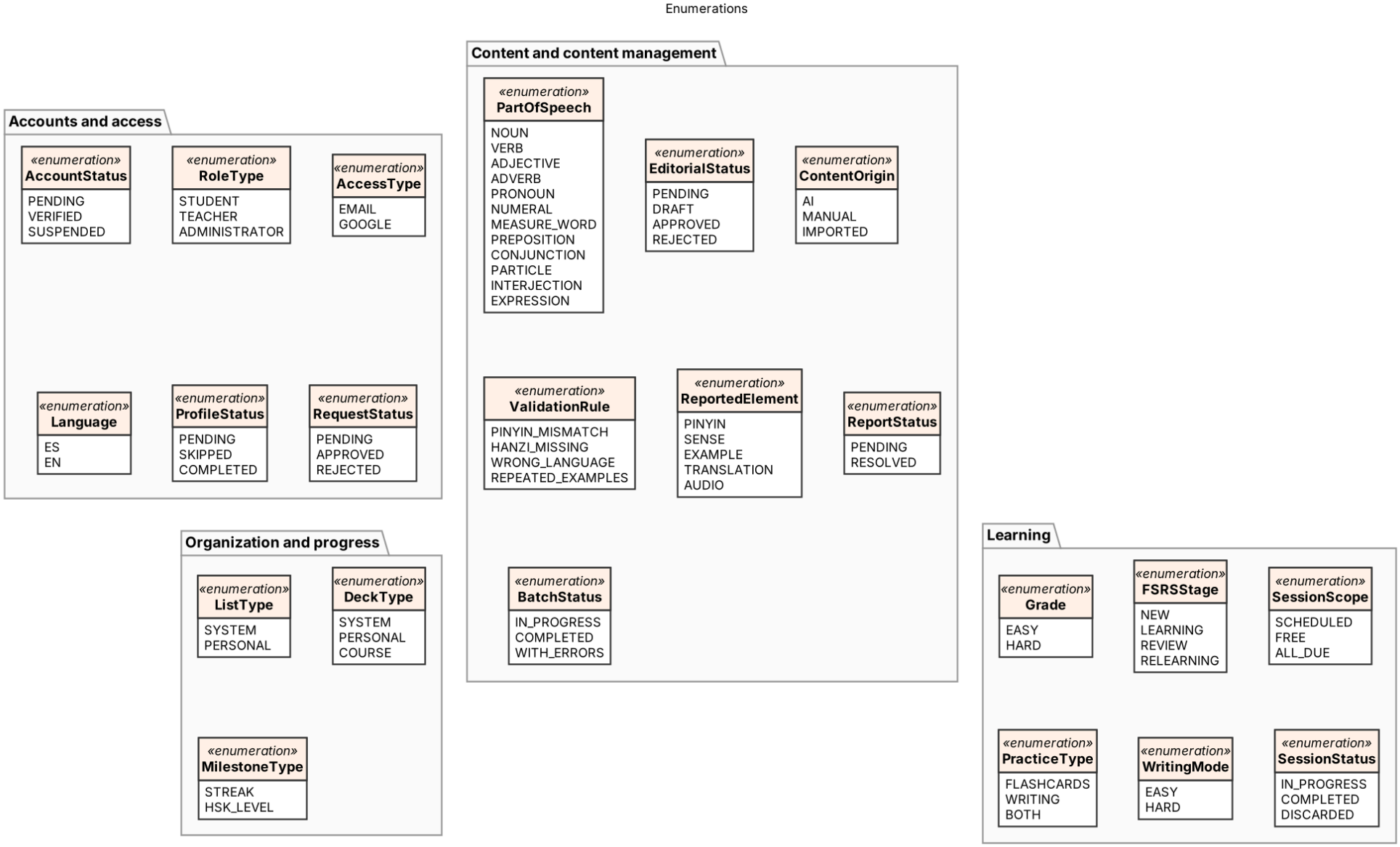
Progress keeps only two records, active days and earned badges; every other number is computed by ProgressCalculator.


Progress package
«control»
SessionQueueBuilder
«use»
Progress
«control»
ProgressCalculator
+currentStreak(u : User) : Integer
+longestStreak(u : User) : Integer
+streakAtRisk(u : User) : Boolean
+learnedWords(u : User) : Integer
+learnedCharacters(u : User) : Integer
+progressByLevel(u : User) : Map<Integer, Float>
+recognitionDue(u : User) : Integer
+writingDue(u : User) : Integer
+newWordsToday(u : User) : Integer
+minutesToday(u : User) : Integer
+awardBadges(u : User) : List<EarnedBadge>
User
1
1
is active on
0..*
earns ▸
0..*
«use»
«create»
«use»
«use»
«use»
ActiveDay
-localDate : Date
EarnedBadge
-date : DateTime
WritingState
PracticeLog
RecognitionState
0..*
awards
1
Badge
-name : String
-milestoneType : MilestoneType
-threshold : Integer
-image : String

- **ProgressCalculator** `«use»`s the active days, both review states and the practice logs, and `«create»`s an EarnedBadge when a milestone is reached (RN-06 to RN-11, RN-45).
- **ActiveDay** is owned by User by composition: deleting the account deletes the days (RN-23.1).

9. Enumerations

The 22 enumerations fix the valid values of every status and type in the model, grouped by the package that uses them.


Enumerations
Accounts and access
«enumeration»
AccountStatus
PENDING
VERIFIED
SUSPENDED
«enumeration»
RoleType
STUDENT
TEACHER
ADMINISTRATOR
«enumeration»
AccessType
EMAIL
GOOGLE
«enumeration»
Language
ES
EN
«enumeration»
ProfileStatus
PENDING
SKIPPED
COMPLETED
«enumeration»
RequestStatus
PENDING
APPROVED
REJECTED
Content and content management
«enumeration»
PartOfSpeech
NOUN
VERB
ADJECTIVE
ADVERB
PRONOUN
NUMERAL
MEASURE_WORD
PREPOSITION
CONJUNCTION
PARTICLE
INTERJECTION
EXPRESSION
«enumeration»
EditorialStatus
PENDING
DRAFT
APPROVED
REJECTED
«enumeration»
ContentOrigin
AI
MANUAL
IMPORTED
«enumeration»
ValidationRule
PINYIN_MISMATCH
HANZI_MISSING
WRONG_LANGUAGE
REPEATED_EXAMPLES
«enumeration»
ReportedElement
PINYIN
SENSE
EXAMPLE
TRANSLATION
AUDIO
«enumeration»
ReportStatus
PENDING
RESOLVED
«enumeration»
BatchStatus
IN_PROGRESS
COMPLETED
WITH_ERRORS
Organization and progress
«enumeration»
ListType
SYSTEM
PERSONAL
«enumeration»
DeckType
SYSTEM
PERSONAL
COURSE
«enumeration»
MilestoneType
STREAK
HSK_LEVEL
Learning
«enumeration»
Grade
EASY
HARD
«enumeration»
FSRSStage
NEW
LEARNING
REVIEW
RELEARNING
«enumeration»
SessionScope
SCHEDULED
FREE
ALL_DUE
«enumeration»
PracticeType
FLASHCARDS
WRITING
BOTH
«enumeration»
WritingMode
EASY
HARD
«enumeration»
SessionStatus
IN_PROGRESS
COMPLETED
DISCARDED

Grade has only two values, EASY and HARD, as the professor requested. FSRSScheduler maps them to the FSRS grades Good and Again (RN-05.1, RN-05.2).

Relational Database Model

HanziApp's database has 39 tables in PostgreSQL 16, the engine the project is moving to from SQLite. 37 tables come from the class diagram and 2 are technical tables for sign-in links and sessions.

The schema was loaded into a real PostgreSQL 16 server and tested: valid sample data loads, and all 20 invalid inserts in the smoke test (a second approved content per language, a deck with two owners, a second in-progress session, and so on) are rejected by the database. The DDL script and the test are attached as `schema.sql` and `smoke_test.sql`.

1. Conventions and mapping decisions

1.1 Conventions

- **Names:** tables and columns in English snake_case; tables in plural (`users`, `deck_words`); foreign keys as `<entity>_id`.
- **Primary keys:** `id bigint GENERATED ALWAYS AS IDENTITY` for entities, `smallint` for the small catalogs (`roles`, `badges`), and composite keys for link tables (`user_roles`, `deck_words`).
- **Enumerations:** `varchar` plus a `CHECK` with the allowed values, instead of PostgreSQL `ENUM` types. They map directly to Django `choices` and are easy to extend in a migration.
- **Dates:** every instant is `timestamptz`, stored in UTC. The only local date is `active_days.local_date`, computed with the user's time zone when the session completes (RN-10.3).
- **Lengths:** text limits come from the user stories (name 50, list and deck 60, report 500).
- **Normal form:** the schema is in third normal form. For example, the class diagram's `Translation.language` is not stored, because it is already determined by `language_contents.language`.

1.2 From the class diagram to tables

Class diagram element	Relational mapping	Example
Concrete entity class (35)	One table each	Word → `words`
Abstract class with subclasses	One table per concrete subclass, with the inherited columns	ReviewState → `recognition_states` and `writing_states`
TranslatableElement (Sense or Example)	Exclusive arc: two nullable foreign keys plus `CHECK (num_nonnulls(sense_id, example_id) = 1)`	`translations`

Many-to-many without data	Link table with a composite key	User–Role → `user_roles`
Intermediate class with its own data	Link table with its own columns	ListDeck → `list_decks.position`
One-to-zero-or-one	Shared primary key	`learner_profiles.user_id` is both PK and FK
Control class and `<<datatype>>`	No table: they hold no persistent data	ProgressCalculator, DeckSummary
Derived attribute	Not stored; computed by query	Streak, learned words, level progress

1.3 What happens on delete

Policy	Used for	Reason
`CASCADE`	Everything a user owns: lists, decks, states, sessions, enrollments	Deleting an account erases its data and progress (RN-23.1)
`RESTRICT`	`words` and `characters` referenced by progress or decks	Words are deactivated, never deleted (RN-27.1)
`SET NULL`	Authorship references: audit log, reports, batches, imports, a session's deck	History survives when an account or deck is deleted (RN-43)

1.4 How to read the ER diagrams

The diagrams use crow's foot notation. A bullet (●) marks a `NOT NULL` column, and «PK», «FK» and «UQ» mark primary, foreign and unique keys. `||` means exactly one, `|o` zero or

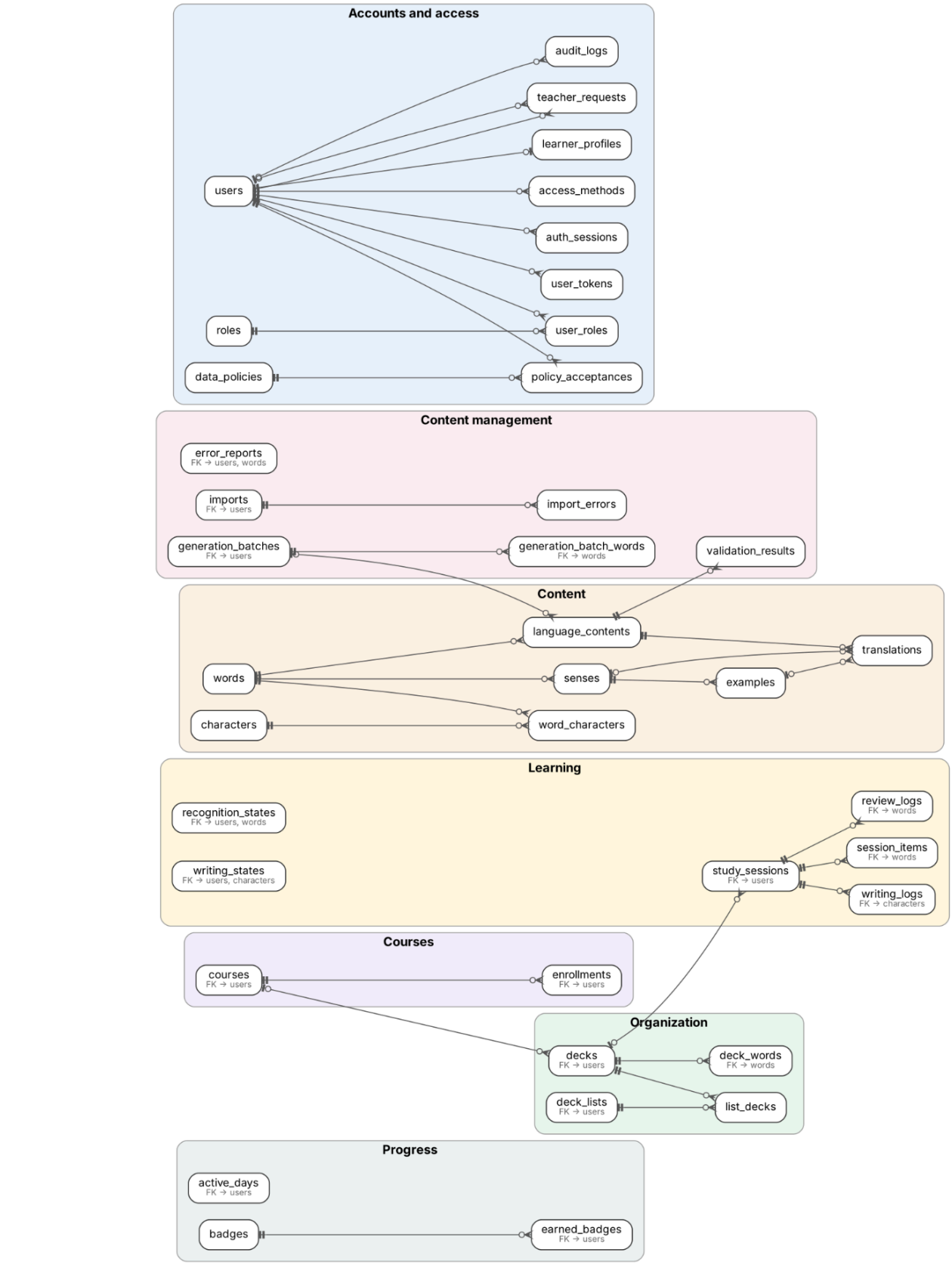
one, and `o{` zero or many. Tables in grey belong to another package. Each line is labeled with the relationship's name, read from the parent table to the child table, as listed in section 3.

2. Overview

The overview shows the 39 tables grouped by package. Foreign keys inside a package, and between packages other than to `users`, `words` and `characters`, are drawn as lines. Foreign keys to those three hub tables from another package are written under the table name (for example, *FK → users, words*), so no long lines cross the picture; each package diagram below draws them in full.

HanziApp · Relational model overview

"FK → users, words" lists foreign keys to tables in another package (see the package diagrams).
|| exactly one |o zero or one o< zero or many



3. Relationships

The schema has 54 relationships, one per foreign key. Each one is listed below with the parent table, its name read from parent to child, the child table, the foreign-key column, its cardinality and what happens to the child rows when the parent row is deleted. The tables are grouped by the package of the child table, and the R numbers match the order of the table definitions.

Cardinality reads *parent : child*. `1 : 0..*` means each child row has exactly one parent and a parent can have any number of children; `0..1` on the parent side means the foreign key is optional; `0..1` on the child side means at most one child per parent.

Accounts and access

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R1	`users`	holds	`user_roles`	`user_id`	1 : 0..*	cascade
R2	`roles`	assigned in	`user_roles`	`role_id`	1 : 0..*	restrict
R3	`users`	signs in with	`access_methods`	`user_id`	1 : 0..*	cascade
R4	`users`	gives	`policy_acceptances`	`user_id`	1 : 0..*	cascade
R5	`data_policies`	accepted in	`policy_acceptances`	`policy_id`	1 : 0..*	restrict
R6	`users`	answers	`learner_profiles`	`user_id`	1 : 0..1	cascade

R7	users	submits	teacher_requests	user_id	1 : 0..*	cascade
R8	users	decides	teacher_requests	decided_by	0..1 : 0..*	set null
R9	users	performs	audit_logs	actor_id	0..1 : 0..*	set null
R10	users	receives	user_tokens	user_id	1 : 0..*	cascade
R11	users	opens	auth_sessions	user_id	1 : 0..*	cascade

Content

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R12	words	is made of	word_characters	word_id	1 : 0..*	cascade
R13	characters	appears in	word_characters	character_id	1 : 0..*	restrict
R14	words	has	senses	word_id	1 : 0..*	cascade

R15	senses	illustrated by	examples	sense_id	1 : 0..*	cascade
R16	words	localized as	language_contents	word_id	1 : 0..*	cascade
R17	generation_batches	produces	language_contents	generation_batch_id	0..1 : 0..*	set null
R18	language_contents	groups	translations	language_content_id	1 : 0..*	cascade
R19	senses	translated as	translations	sense_id	0..1 : 0..*	cascade
R20	examples	translated as	translations	example_id	0..1 : 0..*	cascade

R19 and R20 form an exclusive arc: each translation has exactly one of the two foreign keys.

Content management

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R21	language_contents	validated by	validation_results	language_content_id	1 : 0..*	cascade
R22	words	reported in	error_reports	word_id	1 : 0..*	cascade

R23	users	reports	error_reports	reported_by	0..1 : 0..*	set null
R24	users	resolves	error_reports	resolved_by	0..1 : 0..*	set null
R25	users	starts	generation_batches	created_by	0..1 : 0..*	set null
R26	generation_batches	processes	generation_batch_words	batch_id	1 : 0..*	cascade
R27	words	processed in	generation_batch_words	word_id	1 : 0..*	cascade
R28	users	performs	imports	created_by	0..1 : 0..*	set null
R29	imports	reports	import_errors	import_id	1 : 0..*	cascade

Organization

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R30	users	owns	deck_lists	owner_id	0..1 : 0..*	cascade
R31	users	owns	decks	owner_id	0..1 : 0..*	cascade

R32	courses	owns	decks	course_id	0..1 : 0..*	cascade
R33	deck_lists	contains	list_decks	list_id	1 : 0..*	cascade
R34	decks	placed in	list_decks	deck_id	1 : 0..*	cascade
R35	decks	contains	deck_words	deck_id	1 : 0..*	cascade
R36	words	appears in	deck_words	word_id	1 : 0..*	restrict

R31 and R32 depend on the deck type: a personal deck uses R31, a course deck uses R32, and a system deck uses neither.

Courses

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R37	users	teaches	courses	teacher_id	1 : 0..*	cascade
R38	users	enrolls in	enrollments	student_id	1 : 0..*	cascade
R39	courses	has	enrollments	course_id	1 : 0..*	cascade

Learning

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R40	users	has	recognition_states	user_id	1 : 0..*	cascade
R41	words	tracked by	recognition_states	word_id	1 : 0..*	restrict
R42	users	has	writing_states	user_id	1 : 0..*	cascade
R43	characters	tracked by	writing_states	character_id	1 : 0..*	restrict
R44	users	studies in	study_sessions	user_id	1 : 0..*	cascade
R45	decks	covered by	study_sessions	deck_id	0..1 : 0..*	set null
R46	study_sessions	queues	session_items	session_id	1 : 0..*	cascade
R47	words	presented in	session_items	word_id	1 : 0..*	restrict

R48	study_sessions	records	review_logs	session_id	1 : 0..*	cascade
R49	words	graded in	review_logs	word_id	1 : 0..*	restrict
R50	study_sessions	records	writing_logs	session_id	1 : 0..*	cascade
R51	characters	practiced in	writing_logs	character_id	1 : 0..*	restrict

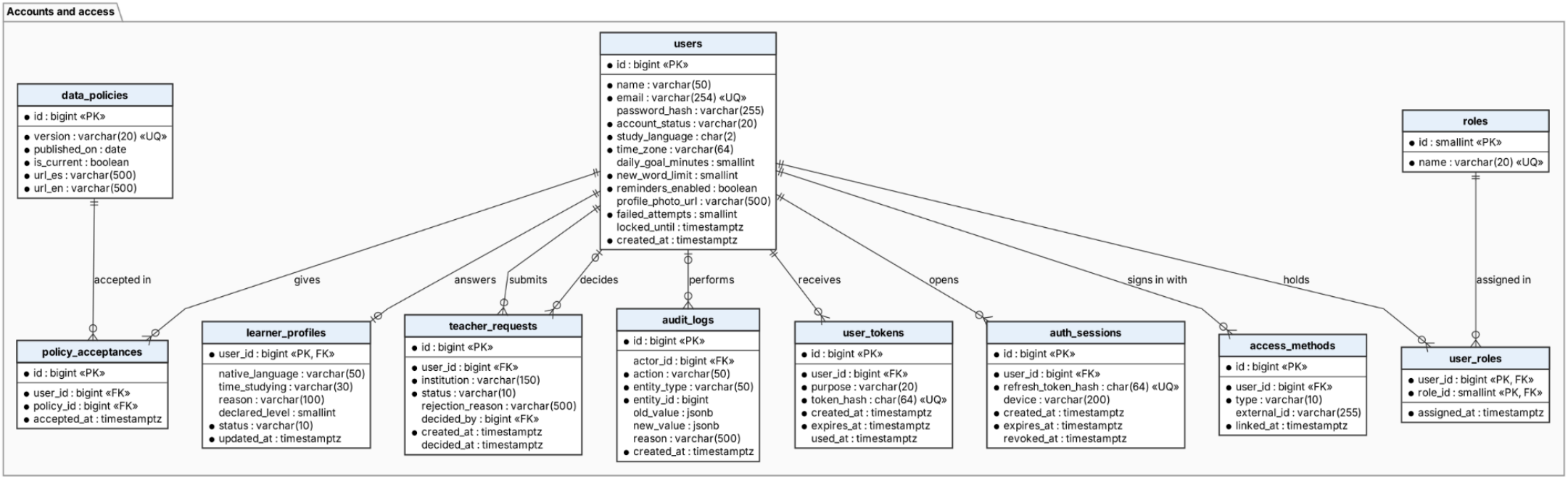
Progress

#	Parent	Relationship	Child	Foreign key	Cardinality	On delete
R52	users	is active on	active_days	user_id	1 : 0..*	cascade
R53	users	earns	earned_badges	user_id	1 : 0..*	cascade
R54	badges	awarded in	earned_badges	badge_id	1 : 0..*	restrict

4. Accounts and access

Eleven tables hold the account, how the user signs in, what they accepted and the administrative trail.

Accounts and access tables



`users`

One row per account, with its identity, settings and status. Class: User.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`name`	varchar(50)	no		`length(trim(name)) > 0`
`email`	varchar(254)	no	UQ	check email = lower(email), so it is always stored in lowercase (RN-19.1)
`password_hash`	varchar(255)	yes		NULL for Google-only accounts
`account_status`	varchar(20)	no		default 'pending'; pending · verified · suspended
`study_language`	char(2)	no		default 'es'; es · en
`time_zone`	varchar(64)	no		default 'America/Bogota'
`daily_goal_minutes`	smallint	yes		`daily_goal_minutes BETWEEN 5 AND 60`
`new_word_limit`	smallint	no		default 10; `new_word_limit BETWEEN 0 AND 50`

`reminders_enabled`	boolean	no		default true
`profile_photo_url`	varchar(500)	yes		
`failed_attempts`	smallint	no		default 0; `failed_attempts >= 0`
`locked_until`	timestamptz	yes		
`created_at`	timestamptz	no		default now()

`roles`

Catalog of the three roles: student, teacher and administrator. Class: Role.

Column	Type	Null	Key	Rules and notes
`id`	smallint	no	PK	
`name`	varchar(20)	no	UQ	student · teacher · administrator

`user_roles`

Roles held by each user. A user can hold several (RN-18.1). Class: User–Role association.

Column	Type	Null	Key	Rules and notes

`user_id`	bigint	no	PK, FK → users	on delete cascade
`role_id`	smallint	no	PK, FK → roles	on delete restrict
`assigned_at`	timestamptz	no		default now()

- Composite primary key (user_id, role_id).

`access_methods`

How a user signs in: email and password, Google, or both (RN-19.2). Class: AccessMethod.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`user_id`	bigint	no	FK → users	on delete cascade
`type`	varchar(10)	no		email · google
`external_id`	varchar(255)	yes		Google subject ID
`linked_at`	timestamptz	no		default now()

- Unique (user_id, type).
- Unique (type, external_id) where `external_id IS NOT NULL`.
- Check: `(type = 'google') = (external_id IS NOT NULL)`.

`data_policies`

Published versions of the data processing policy (RNF-25). Class: DataPolicy.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`version`	varchar(20)	no	UQ	
`published_on`	date	no		
`is_current`	boolean	no		default false
`url_es`	varchar(500)	no		
`url_en`	varchar(500)	no		

- Unique (is_current) where `is_current`: only one version is current.

`policy_acceptances`

Each policy version a user accepted, with date and time (RN-17.2, RNF-26). Class: PolicyAcceptance.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity

`user_id`	bigint	no	FK → users	on delete cascade
`policy_id`	bigint	no	FK → data_policies	on delete restrict
`accepted_at`	timestamptz	no		default now()

- Unique (user_id, policy_id).

`learner_profiles`

Answers to the onboarding questionnaire; every answer is optional (RNF-30). Class: LearnerProfile.

Column	Type	Null	Key	Rules and notes
`user_id`	bigint	no	PK, FK → users	on delete cascade
`native_language`	varchar(50)	yes		
`time_studying`	varchar(30)	yes		
`reason`	varchar(100)	yes		
`declared_level`	smallint	yes		`declared_level BETWEEN 1 AND 6`

`status`	varchar(10)	no		default 'pending'; pending · skipped · completed
`updated_at`	timestampt z	no		default now()

`teacher_requests`

Requests for the teacher role and their outcome. Class: TeacherRequest.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`user_id`	bigint	no	FK → users	on delete cascade
`institution`	varchar(150)	no		`length(trim(institution)) > 0`
`status`	varchar(10)	no		default 'pending'; pending · approved · rejected
`rejection_reason`	varchar(500)	yes		
`decided_by`	bigint	yes	FK → users	on delete set null
`created_at`	timestampt z	no		default now()

`decided_at`	timestampt z	yes	

- Unique (user_id) where `status = 'pending'`: one pending request per user (RN-48.1).
- Check: `status <> 'rejected' OR rejection_reason IS NOT NULL` (RN-50).

`audit_logs`

Administrative actions with author, date and before/after values (RN-49). Class: AuditLog.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`actor_id`	bigint	yes	FK → users	on delete set null
`action`	varchar(50)	no		
`entity_type`	varchar(50)	no		
`entity_id`	bigint	no		
`old_value`	jsonb	yes		
`new_value`	jsonb	yes		

`reason`	varchar(500)	yes		
`created_at`	timestamptz	no		default now()

- Index (entity_type, entity_id) and index (created_at).

`user_tokens`

Single-use email verification and password reset links (RN-16.2, RNF-23). Technical table, not in the class diagram.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`user_id`	bigint	no	FK → users	on delete cascade
`purpose`	varchar(20)	no		email_verification · password_reset
`token_hash`	char(64)	no	UQ	SHA-256 of the token; the token itself is never stored
`created_at`	timestamptz	no		default now()
`expires_at`	timestamptz	no		

`used_at`	timestampt z	yes

- Check: `expires_at > created_at`.
- Index (user_id, purpose).

`auth_sessions`

Signed-in devices; revoking a row ends that session (RF-13, RNF-23, RN-22.2). Technical table, not in the class diagram.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`user_id`	bigint	no	FK → users	on delete cascade
`refresh_token_hash`	char(64)	no	UQ	
`device`	varchar(200)	yes		
`created_at`	timestamptz	no		default now()
`expires_at`	timestamptz	no		
`revoked_at`	timestamptz	yes		

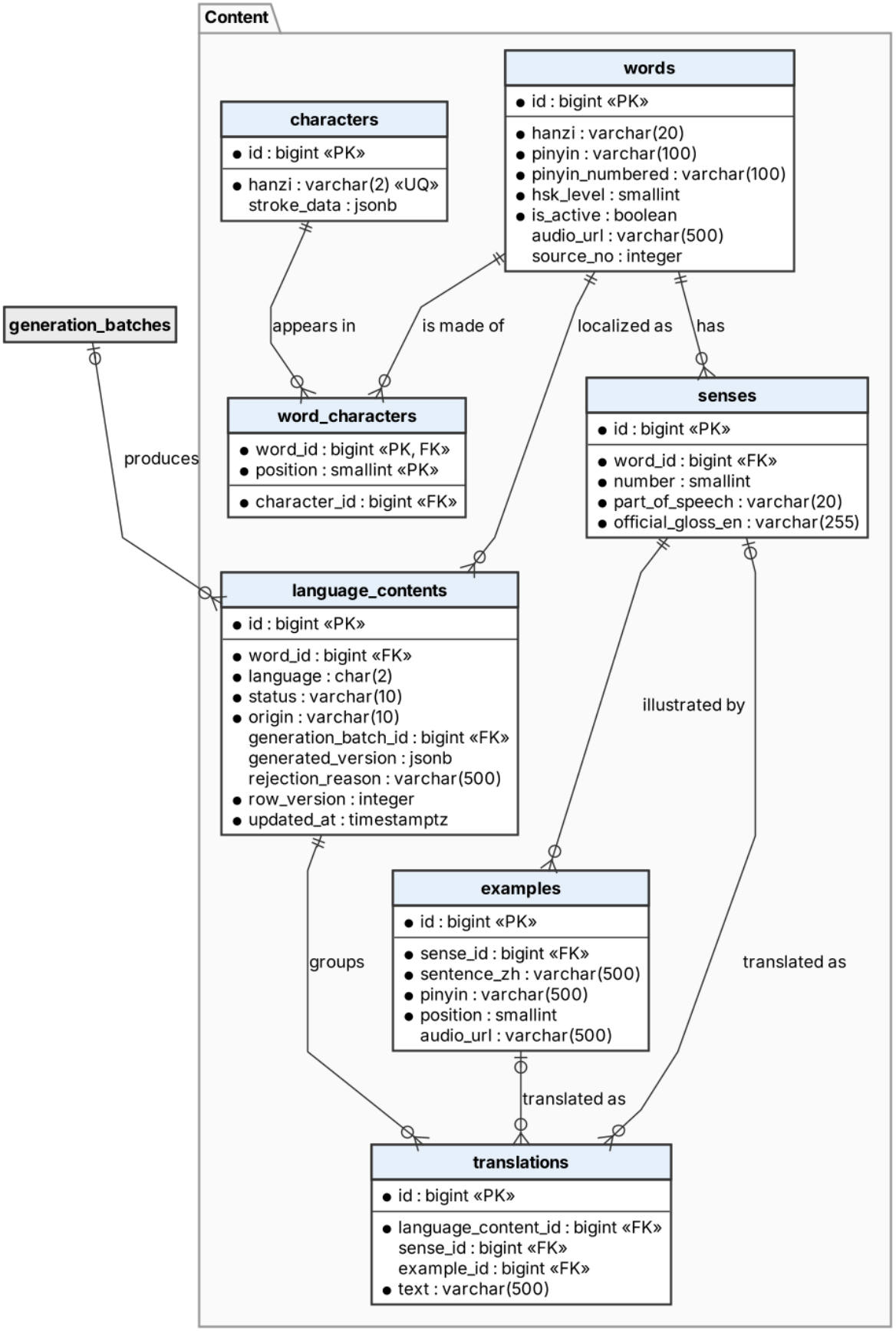
- Index (user_id) where `revoked_at IS NULL`.

5. Content and content management

5.1 Content

Seven tables separate what is shared across languages (`words`, `characters`, `senses`, `examples`) from what changes per language (`language_contents` and its `translations`).

Content tables



words

HSK 3.0 vocabulary, one row per hanzi and reading (RN-26.1). Class: Word.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
hanzi	varchar(20)	no		
pinyin	varchar(100)	no		With tone marks
pinyin_numbered	varchar(100)	no		Derived from pinyin, for search
hsk_level	smallint	no		hsk_level BETWEEN 1 AND 6
is_active	boolean	no		default true
audio_url	varchar(500)	yes		
source_no	integer	yes		Row number in the official list

- Unique (hanzi, pinyin): 还 hái and 还 huán are two rows.
- Index (hsk_level) and index (pinyin_numbered).

characters

Distinct characters, with their stroke data for HanziWriter. Class: Character.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`hanzi`	varchar(2)	no	UQ	`char_length(hanzi) = 1`
`stroke_data`	jsonb	yes		

`word_characters`

Characters that make up each word, in order (RN-28). Class: WordComposition.

Column	Type	Null	Key	Rules and notes
`word_id`	bigint	no	PK, FK → words	on delete cascade
`position`	smallint	no	PK	`position >= 1`
`character_id`	bigint	no	FK → characters	on delete restrict

- Composite primary key (word_id, position).
- Index (character_id), for the character page (RF-40).

`senses`

Senses of each word, shared across languages (RN-24). Class: Sense.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`word_id`	bigint	no	FK → words	on delete cascade
`number`	smallint	no		`number >= 1`
`part_of_speech`	varchar(20)	no		noun · verb · adjective · adverb · pronoun · numeral · measure_word · preposition · conjunction · particle · interjection · expression
`official_gloss_en`	varchar(255)	no		

- Unique (word_id, number).

examples

Chinese example sentences of each sense, shared across languages. Class: Example.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`sense_id`	bigint	no	FK → senses	on delete cascade

`sentence_zh`	varchar(500)	no		
`pinyin`	varchar(500)	no		
`position`	smallint	no		`position >= 1`
`audio_url`	varchar(500)	yes		

- Unique (sense_id, position).

`language_contents`

A word's content in one language, with its editorial status (RN-25.1). Class: LanguageContent.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`word_id`	bigint	no	FK → words	on delete cascade
`language`	char(2)	no		es · en
`status`	varchar(10)	no		default 'draft'; pending · draft · approved · rejected
`origin`	varchar(10)	no		ai · manual · imported

`generation_batch_id`	bigint	yes	FK → generation_batches	on delete set null
`generated_version`	jsonb	yes		AI output before edits (HU-104)
`rejection_reason`	varchar(500)	yes		
`row_version`	integer	no		default 1; optimistic locking (HU-97)
`updated_at`	timestamptz	no		default now()

- Unique (word_id, language) where `status = 'approved'`: at most one approved content per word and language (RN-25.2).
- Check: `status <> 'rejected' OR rejection_reason IS NOT NULL` (RN-50).
- Index (word_id, language).

translations

Translation of a sense or of an example, never both (exclusive arc). Class: Translation.

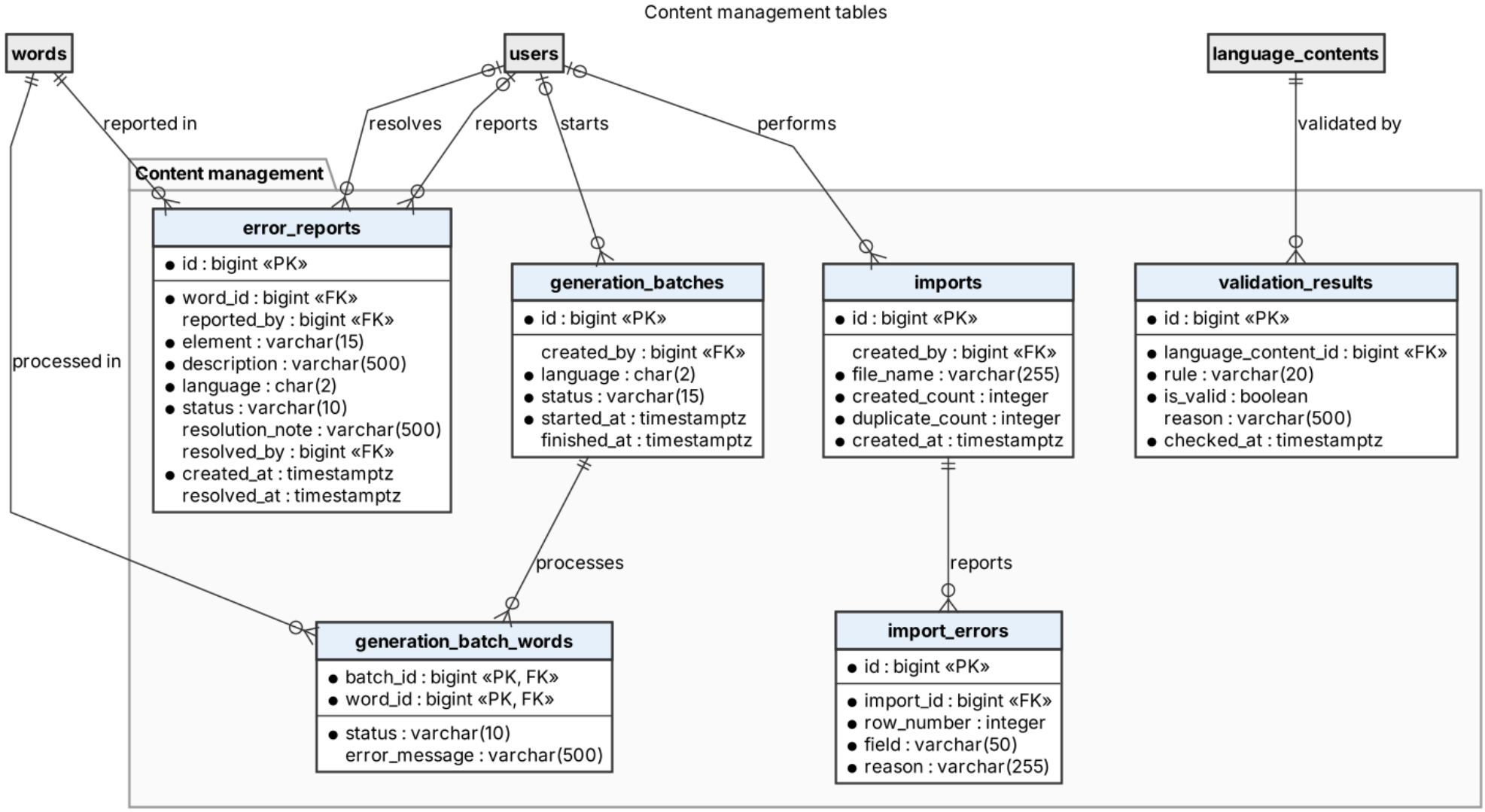
Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`language_content_id`	bigint	no	FK → language_contents	on delete cascade

`sense_id`	bigint	yes	FK → senses	on delete cascade
`example_id`	bigint	yes	FK → examples	on delete cascade
`text`	varchar(500)	no		`length(trim(text)) > 0`

- Unique (language_content_id, sense_id) where `sense_id IS NOT NULL`.
- Unique (language_content_id, example_id) where `example_id IS NOT NULL`.
- Check: `num_nonnulls(sense_id, example_id) = 1`.

5.2 Content management

Six tables record the administrator's work: validation results, user reports, AI generation batches and bulk imports.


Content management tables
words
users
language_contents
reported in
resolves
reports
starts
performs
validated by
Content management
error_reports
id : bigint «PK»
word_id : bigint «FK»
reported_by : bigint «FK»
element : varchar(15)
description : varchar(500)
language : char(2)
status : varchar(10)
resolution_note : varchar(500)
resolved_by : bigint «FK»
created_at : timestamptz
resolved_at : timestamptz
generation_batches
id : bigint «PK»
created_by : bigint «FK»
language : char(2)
status : varchar(15)
started_at : timestamptz
finished_at : timestamptz
imports
id : bigint «PK»
created_by : bigint «FK»
file_name : varchar(255)
created_count : integer
duplicate_count : integer
created_at : timestamptz
validation_results
id : bigint «PK»
language_content_id : bigint «FK»
rule : varchar(20)
is_valid : boolean
reason : varchar(500)
checked_at : timestamptz
processed in
processes
reports
generation_batch_words
batch_id : bigint «PK, FK»
word_id : bigint «PK, FK»
status : varchar(10)
error_message : varchar(500)
import_errors
id : bigint «PK»
import_id : bigint «FK»
row_number : integer
field : varchar(50)
reason : varchar(255)

validation_results

Outcome of each automatic check on a draft (RF-103). Class: ValidationResult.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
language_content_id	bigint	no	FK → language_contents	on delete cascade
rule	varchar(20)	no		pinyin_mismatch · hanzi_missing · wrong_language · repeated_examples
is_valid	boolean	no		
reason	varchar(500)	yes		
checked_at	timestamptz	no		default now()

- Check: `is_valid OR reason IS NOT NULL`: every failure has a reason.
- Index (language_content_id).

error_reports

Content errors reported by users (RF-42, RF-107). Class: ErrorReport.

Column	Type	Null	Key	Rules and notes

`id`	bigint	no	PK	identity
`word_id`	bigint	no	FK → words	on delete cascade
`reported_by`	bigint	yes	FK → users	on delete set null
`element`	varchar(15)	no		pinyin · sense · example · translation · audio
`description`	varchar(500)	no		`length(trim(description)) > 0`
`language`	char(2)	no		es · en
`status`	varchar(10)	no		default 'pending'; pending · resolved
`resolution_note`	varchar(500)	yes		
`resolved_by`	bigint	yes	FK → users	on delete set null
`created_at`	timestamptz	no		default now()
`resolved_at`	timestamptz	yes		

- Index (word_id) where `status = 'pending'`, for the grouped review queue (HU-107).
- Index (reported_by, created_at), for the 10-reports-per-hour limit (HU-42).

generation_batches

AI generation runs of up to 100 words (RN-51.1). Class: GenerationBatch.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
created_by	bigint	yes	FK → users	on delete set null
language	char(2)	no		es · en
status	varchar(15)	no		default 'in_progress'; in_progress · completed · with_errors
started_at	timestamptz	no		default now()
finished_at	timestamptz	yes		

generation_batch_words

Words processed by each batch, with their individual result so failures can be retried (HU-102). Class: GenerationBatch–Word association.

Column	Type	Null	Key	Rules and notes

batch_id	bigint	no	PK, FK → generation_batches	on delete cascade
word_id	bigint	no	PK, FK → words	on delete cascade
status	varchar(10)	no		default 'pending'; pending · done · failed
error_message	varchar(500)	yes		

- Composite primary key (batch_id, word_id).

imports

Bulk vocabulary imports and their summary (RF-100). Class: Import.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
created_by	bigint	yes	FK → users	on delete set null
file_name	varchar(255)	no		
created_count	integer	no		default 0
duplicate_count	integer	no		default 0

`created_at`	timestamptz	no		default now()

- The number of rows with errors is not stored: it is the count of `import_errors` for the import, which keeps the table in third normal form.

`import_errors`

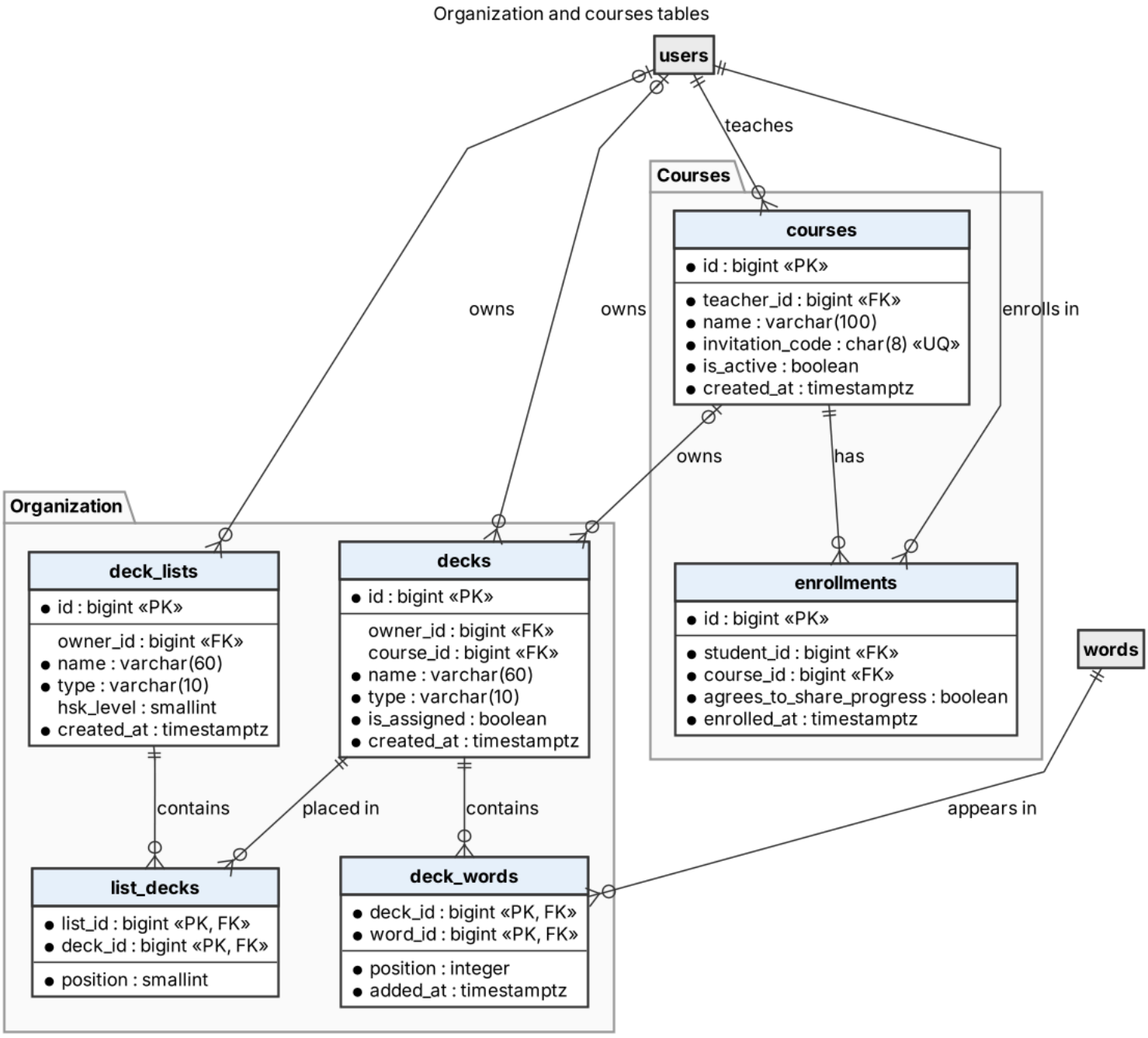
Rows that could not be imported, with field and reason (RF-101). Class: ImportError.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`import_id`	bigint	no	FK → imports	on delete cascade
`row_number`	integer	no		`row_number >= 1`
`field`	varchar(50)	no		
`reason`	varchar(255)	no		

- Index (import_id).

6. Organization and courses

Six tables organize vocabulary: lists contain decks through `list_decks`, decks contain words through `deck_words`, and courses own their decks and their enrollments.


Organization and courses tables
users
teaches
Courses
courses
id : bigint «PK»
teacher_id : bigint «FK»
name : varchar(100)
invitation_code : char(8) «UQ»
is_active : boolean
created_at : timestamptz
owns
owns
enrolls in
owns
has
Organization
deck_lists
id : bigint «PK»
owner_id : bigint «FK»
name : varchar(60)
type : varchar(10)
hsk_level : smallint
created_at : timestamptz
decks
id : bigint «PK»
owner_id : bigint «FK»
course_id : bigint «FK»
name : varchar(60)
type : varchar(10)
is_assigned : boolean
created_at : timestamptz
enrollments
id : bigint «PK»
student_id : bigint «FK»
course_id : bigint «FK»
agrees_to_share_progress : boolean
enrolled_at : timestamptz
words
contains
placed in
contains
appears in
list_decks
list_id : bigint «PK, FK»
deck_id : bigint «PK, FK»
position : smallint
deck_words
deck_id : bigint «PK, FK»
word_id : bigint «PK, FK»
position : integer
added_at : timestamptz

The six system HSK lists are created by the script itself, as catalog data.

`deck_lists`

System HSK lists and users' own lists. Class: DeckList.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`owner_id`	bigint	yes	FK → users	on delete cascade; NULL for system lists
`name`	varchar(60)	no		`length(trim(name)) > 0`
`type`	varchar(10)	no		system · personal
`hsk_level`	smallint	yes		Only for system lists
`created_at`	timestamptz	no		default now()

- Unique (owner_id, name) where `owner_id IS NOT NULL` (RN-30.1).
- Unique (hsk_level) where `type = 'system'`: one system list per level.
- Check: a system list has no owner and a level from 1 to 6; a personal list has an owner and no level.

`decks`

System, personal and course decks; each type has a single kind of owner. Class: Deck.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`owner_id`	bigint	yes	FK → users	on delete cascade; only for personal decks
`course_id`	bigint	yes	FK → courses	on delete cascade; only for course decks
`name`	varchar(60)	no		`length(trim(name)) > 0`
`type`	varchar(10)	no		system · personal · course
`is_assigned`	boolean	no		default false; course decks visible to students
`created_at`	timestamptz	no		default now()

- Unique (course_id, name) where `course_id IS NOT NULL` (RN-30.2).
- Check: a system deck has no owner and no course, a personal deck has only an owner, and a course deck has only a course.
- Check: `is_assigned = false OR type = 'course'`.
- Index (owner_id).

`list_decks`

Decks in each list, with their position in that list (RN-29.1). Class: ListDeck.

Column	Type	Null	Key	Rules and notes
`list_id`	bigint	no	PK, FK → deck_lists	on delete cascade
`deck_id`	bigint	no	PK, FK → decks	on delete cascade
`position`	smallint	no		`position >= 1`

- Composite primary key (list_id, deck_id): a deck can be in several lists, once in each.
- Unique (list_id, position), checked at commit so decks can be reordered in a single transaction.
- Index (deck_id).

`deck_words`

Words in each deck, in order (RN-03). Class: DeckWord.

Column	Type	Null	Key	Rules and notes
`deck_id`	bigint	no	PK, FK → decks	on delete cascade
`word_id`	bigint	no	PK, FK → words	on delete restrict
`position`	integer	no		`position >= 1`

`added_at`	timestamptz	no		default now()

- Composite primary key (deck_id, word_id): a word appears at most once per deck (HU-41).
- Index (word_id).

`courses`

Courses created by teachers. Class: Course.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`teacher_id`	bigint	no	FK → users	on delete cascade
`name`	varchar(100)	no		`length(trim(name)) > 0`
`invitation_code`	char(8)	no	UQ	
`is_active`	boolean	no		default true
`created_at`	timestamptz	no		default now()

- Index (teacher_id).

`enrollments`

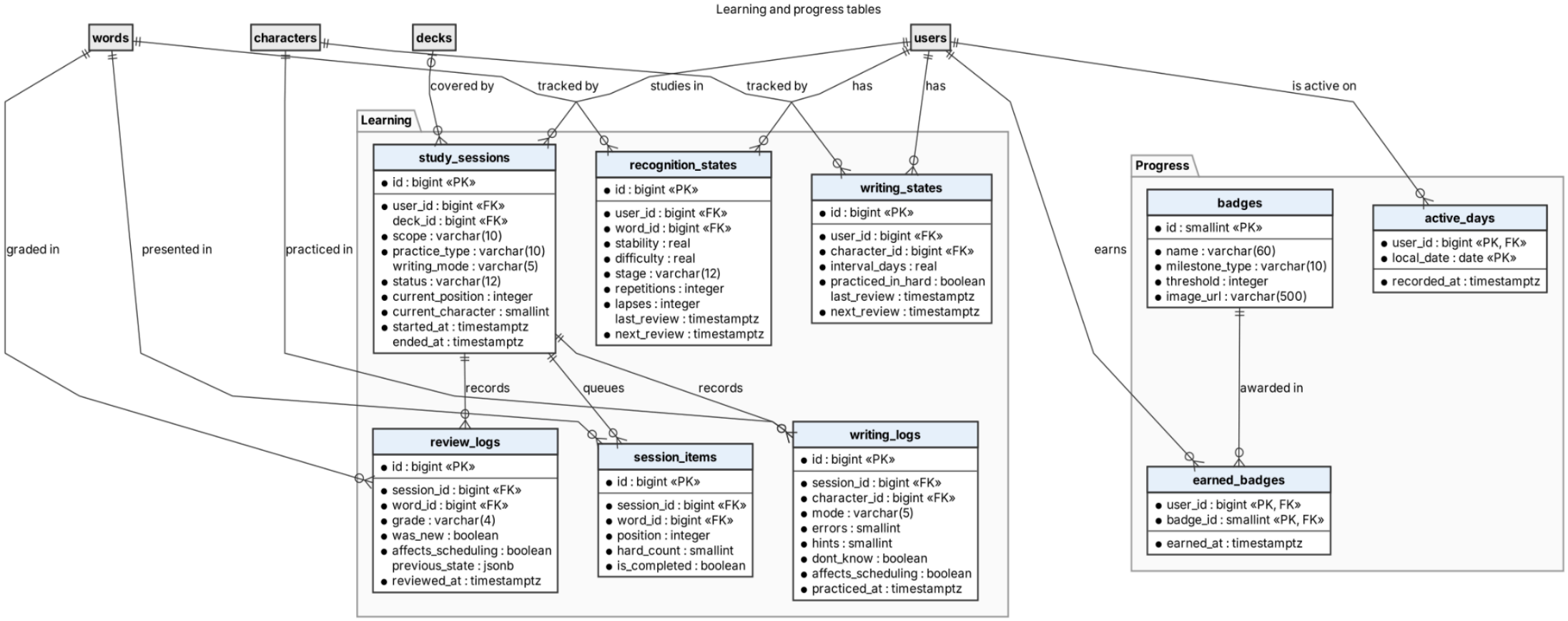
Students enrolled in each course, with their consent (RN-46.1). Class: Enrollment.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`student_id`	bigint	no	FK → users	on delete cascade
`course_id`	bigint	no	FK → courses	on delete cascade
`agrees_to_share_progress`	boolean	no		must be true
`enrolled_at`	timestamptz	no		default now()

- Unique (student_id, course_id).
- Index (course_id).

7. Learning and progress

Nine tables hold the review states, the sessions with their queue and logs, and the progress records. The review states never reference a deck, which is what keeps them global (RN-01, RN-02).


Learning and progress tables
words
characters
decks
users
covered by
tracked by
studies in
tracked by
has
has
is active on
Learning
study_sessions
• id : bigint «PK»
• user_id : bigint «FK»
deck_id : bigint «FK»
• scope : varchar(10)
• practice_type : varchar(10)
writing_mode : varchar(5)
• status : varchar(12)
• current_position : integer
• current_character : smallint
• started_at : timestamptz
ended_at : timestamptz
recognition_states
• id : bigint «PK»
• user_id : bigint «FK»
• word_id : bigint «FK»
• stability : real
• difficulty : real
• stage : varchar(12)
• repetitions : integer
• lapses : integer
last_review : timestamptz
• next_review : timestamptz
writing_states
• id : bigint «PK»
• user_id : bigint «FK»
• character_id : bigint «FK»
• interval_days : real
• practiced_in_hard : boolean
last_review : timestamptz
• next_review : timestamptz
graded in
presented in
practiced in
earns
records
queues
records
review_logs
• id : bigint «PK»
• session_id : bigint «FK»
• word_id : bigint «FK»
• grade : varchar(4)
• was_new : boolean
• affects_scheduling : boolean
previous_state : jsonb
• reviewed_at : timestamptz
session_items
• id : bigint «PK»
• session_id : bigint «FK»
• word_id : bigint «FK»
• position : integer
• hard_count : smallint
• is_completed : boolean
writing_logs
• id : bigint «PK»
• session_id : bigint «FK»
• character_id : bigint «FK»
• mode : varchar(5)
• errors : smallint
• hints : smallint
• dont_know : boolean
• affects_scheduling : boolean
• practiced_at : timestamptz
Progress
badges
• id : smallint «PK»
• name : varchar(60)
• milestone_type : varchar(10)
• threshold : integer
• image_url : varchar(500)
active_days
• user_id : bigint «PK, FK»
• local_date : date «PK»
• recorded_at : timestamptz
awarded in
earned_badges
• user_id : bigint «PK, FK»
• badge_id : smallint «PK, FK»
• earned_at : timestamptz

recognition_states

One FSRS state per user and word, independent of decks (RN-01). Class: RecognitionState.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
user_id	bigint	no	FK → users	on delete cascade
word_id	bigint	no	FK → words	on delete restrict
stability	real	no		stability >= 0
difficulty	real	no		difficulty BETWEEN 1 AND 10
stage	varchar(12)	no		new · learning · review · relearning
repetitions	integer	no		default 0
lapses	integer	no		default 0
last_review	timestamptz	yes		
next_review	timestamptz	no		

- Unique (user_id, word_id): one state per word, whatever the deck.
- Index (user_id, next_review), for the due-reviews queries.

writing_states

One writing state per user and character, independent of words and decks (RN-02). Class: WritingState.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
user_id	bigint	no	FK → users	on delete cascade
character_id	bigint	no	FK → characters	on delete restrict
interval_days	real	no		interval_days >= 0
practiced_in_hard	boolean	no		default false
last_review	timestamptz	yes		
next_review	timestamptz	no		

- Unique (user_id, character_id).
- Index (user_id, next_review).

study_sessions

Study sessions, including in-progress ones that can be resumed (RN-12.1). Class: StudySession.

Column	Type	Null	Key	Rules and notes
id	bigint	no	PK	identity
user_id	bigint	no	FK → users	on delete cascade
deck_id	bigint	yes	FK → decks	on delete set null; NULL in the all-due-reviews session
scope	varchar(10)	no		scheduled · free · all_due
practice_type	varchar(10)	no		flashcards · writing · both
writing_mode	varchar(5)	yes		easy · hard
status	varchar(12)	no		default 'in_progress'; in_progress · completed · discarded
current_position	integer	no		default 1
current_character	smallint	no		default 1
started_at	timestamptz	no		default now()

`ended_at`	timestamptz	yes		

- Unique (user_id, deck_id, practice_type) where `status = 'in_progress'`: one in-progress session per deck and practice type. NULLS NOT DISTINCT makes the rule hold for the all-due-reviews session too, which has no deck (RN-12.2).
- Check: `(practice_type = 'flashcards') = (writing_mode IS NULL)`: a writing mode exists only when there is writing.
- Check: `(status = 'in_progress') = (ended_at IS NULL)`.
- Index (user_id, ended_at), for minutes studied today.

`session_items`

Stored queue of each session, so it can be resumed (RN-04.2). Class: SessionItem.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`session_id`	bigint	no	FK → study_sessions	on delete cascade
`word_id`	bigint	no	FK → words	on delete restrict
`position`	integer	no		`position >= 1`
`hard_count`	smallint	no		default 0; `hard_count BETWEEN 0 AND 3` (RN-36.2)
`is_completed`	boolean	no		default false

- Unique (session_id, word_id).
- Index (session_id, position).

`review_logs`

Every flashcard grade, used for undo, the new-word limit and statistics. Class: ReviewLog.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`session_id`	bigint	no	FK → study_sessions	on delete cascade
`word_id`	bigint	no	FK → words	on delete restrict
`grade`	varchar(4)	no		easy · hard
`was_new`	boolean	no		
`affects_scheduling`	boolean	no		false in free review (RN-40)
`previous_state`	jsonb	yes		State before the grade, for undo (RF-60)
`reviewed_at`	timestamptz	no		default now()

- Index (session_id) and index (word_id).

`writing_logs`

Every character written in a session, with errors and hints. Class: WritingLog.

Column	Type	Null	Key	Rules and notes
`id`	bigint	no	PK	identity
`session_id`	bigint	no	FK → study_sessions	on delete cascade
`character_id`	bigint	no	FK → characters	on delete restrict
`mode`	varchar(5)	no		easy · hard
`errors`	smallint	no		default 0; `errors >= 0`
`hints`	smallint	no		default 0; `hints >= 0`
`dont_know`	boolean	no		default false
`affects_scheduling`	boolean	no		
`practiced_at`	timestamptz	no		default now()

- Index (session_id) and index (character_id).

`active_days`

Days on which the user completed a session, in their local time zone (RN-10.1). Class: ActiveDay.

Column	Type	Null	Key	Rules and notes
`user_id`	bigint	no	PK, FK → users	on delete cascade
`local_date`	date	no	PK	
`recorded_at`	timestamptz	no		default now()

- Composite primary key (user_id, local_date): a day counts once, however many sessions (RN-11).

`badges`

Catalog of badges and the milestone that unlocks each one. Class: Badge.

Column	Type	Null	Key	Rules and notes
`id`	smallint	no	PK	
`name`	varchar(60)	no		
`milestone_type`	varchar(10)	no		streak · hsk_level
`threshold`	integer	no		`threshold > 0`; days of streak or HSK level

`image_url`	varchar(500)	no

- Unique (milestone_type, threshold).

`earned_badges`

Badges earned by each user, once each (RN-45). Class: EarnedBadge.

Column	Type	Null	Key	Rules and notes
`user_id`	bigint	no	PK, FK → users	on delete cascade
`badge_id`	smallint	no	PK, FK → badges	on delete restrict
`earned_at`	timestamptz	no		default now()

- Composite primary key (user_id, badge_id): a badge cannot be earned twice (RN-45).

8. Rules enforced outside table constraints

Twelve rules, ten of them business rules and two security requirements, span several rows or tables, so a column `CHECK` or a unique key cannot express them. They are enforced in the Django service layer, inside the same transaction as the change (`transaction.atomic()`). The first three could later become triggers if they are broken in practice.

Rule	Why it is not a constraint	How it is enforced

A deck name is unique within each list (RN-30.2)	The name is in `decks`, the membership in `list_decks`	Checked when a deck is created, renamed or added to a list
In a system list, each word is in one deck and belongs to the list's level (RN-31.1, RN-31.2)	Spans `deck_words`, `list_decks`, `deck_lists` and `words`	Checked when an administrator assigns words
A translation belongs to a sense or example of the same word as its content	Spans `translations`, `language_contents`, `senses` and `examples`	Checked when a draft is created or edited
Deleting a list deletes the decks that are in no other list (RN-29.2)	The cascade only removes the `list_decks` links	The service deletes the orphaned personal decks in the same transaction
At least one administrator exists (RN-18.3)	It is a count over `user_roles`	Checked before removing the administrator role
A generation batch has at most 100 words (RN-51.1)	It is a count over `generation_batch_words`	Checked when the batch is created
Five failed attempts in 15 minutes lock the account (RNF-22)	It depends on time	Updates `failed_attempts` and `locked_until` on each attempt
Only the newest reset link is valid (RNF-23)	It compares rows	Issuing a link marks the user's earlier unused links as used

The new-word limit per day (RN-37.1, RN-37.3)	It is a count per local day	Counted from `review_logs.was_new`, excluding free review
Empty system decks are hidden; empty decks cannot be studied (RN-31.3, RN-34)	It depends on `deck_words`	Query condition when listing decks and starting a session
Streak, learned words and level progress (RN-06 to RN-11)	Derived values	Computed by query from `active_days` and the review states
Administrative actions are audited (RN-49)	It spans every administrative table	The service writes `audit_logs` in the same transaction

Mockups

Landing page: Shows the value of HanziApp.

HanziApp

Funciones Niveles HSK

Inicia sesión

Aprender chino nunca fue tan fácil

Aprende vocabulario, practica la escritura y mejora tu habilidad auditiva.

Empezar gratis



Aprende. Practica. Avanza.

PRUÉBALO TÚ MISMO

Una tarjeta de verdad, ahora mismo



HanziApp

Funciones Niveles HSK

Inicia sesión

Todo lo que necesitas para aprender chino

Vocabulario por niveles HSK

Aprende de forma progresiva con contenido organizado por nivel y misiones de 20–30 palabras.

Escritura sin papel

Escribe caracteres directamente en pantalla y practica el orden correcto de los trazos.

Habilidad auditiva

Escucha, identifica y practica el vocabulario que estás aprendiendo.

Sin configuración

Empieza a estudiar en segundos. Sin mazos ni setups tediosos.

En español e inglés

Toda la interfaz en español. Pensado para hispanohablantes, no adaptado.

Crea tus propios mazos

Agrega tus propias palabras y practica con active recall. Tu aprendizaje, a tu manera.

HanziApp

Funciones Niveles HSK

Inicia sesión



Empieza hoy. Es gratis.

Tu camino para aprender chino empieza aquí.

Crear cuenta gratis →

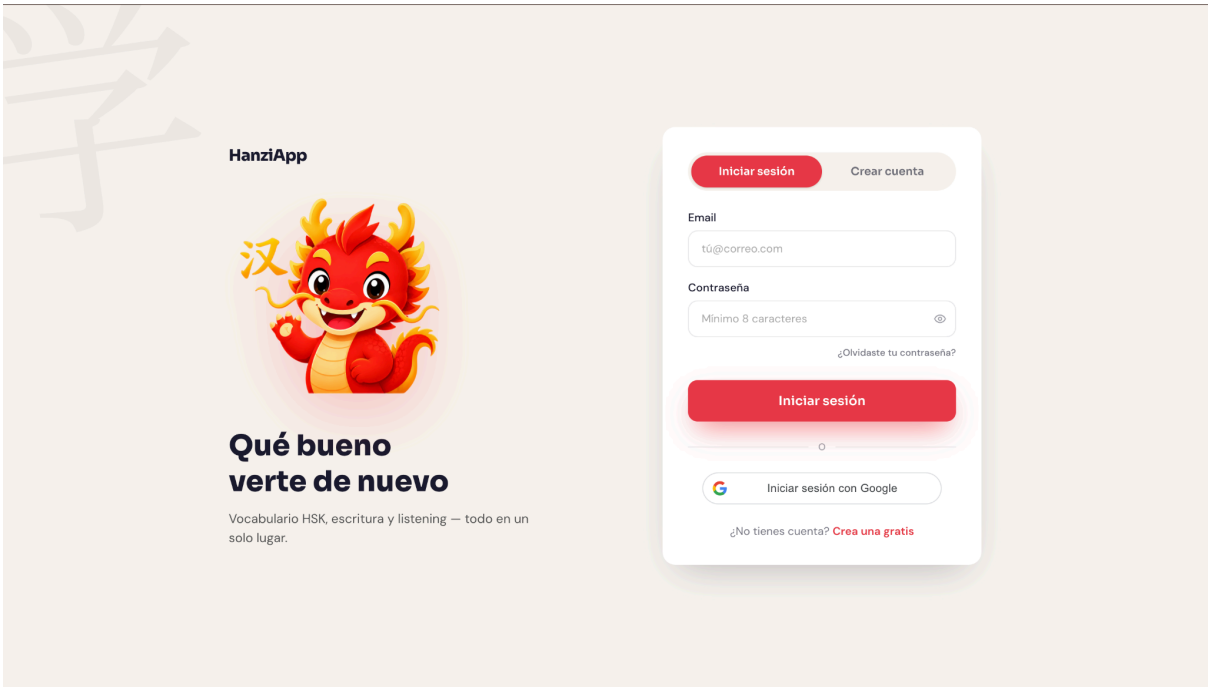
HanziApp

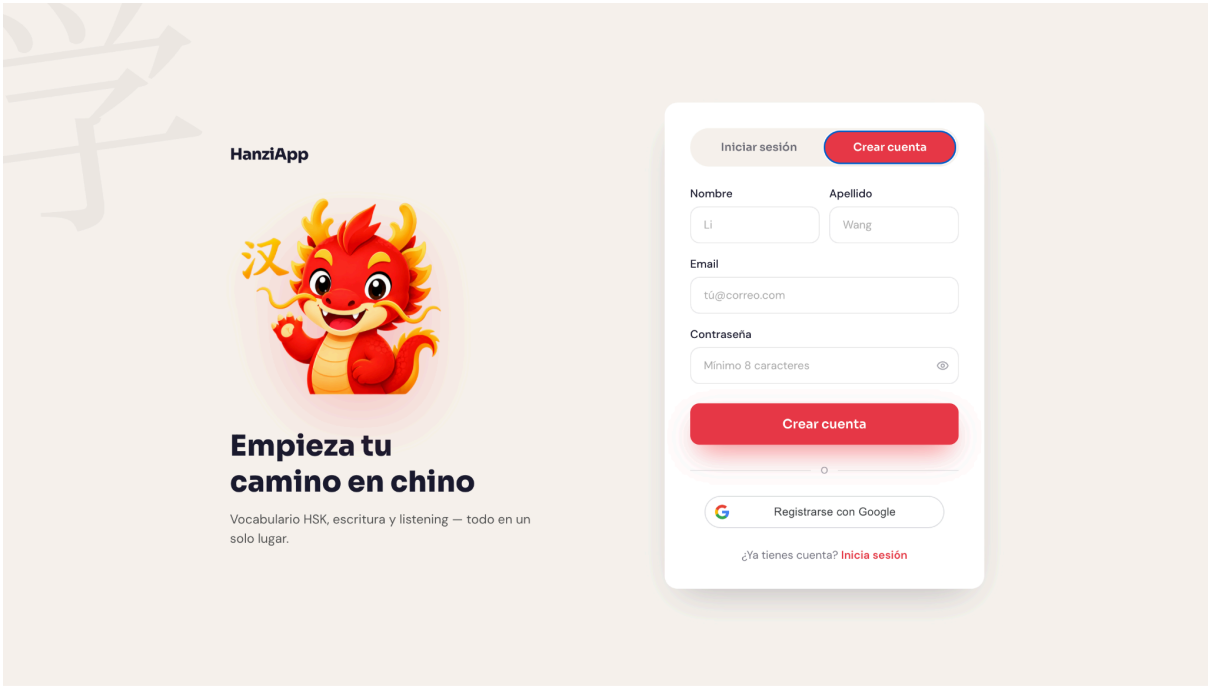
Funciones Niveles HSK Contacto

Hecho con ❤️ en Colombia 🇨🇴

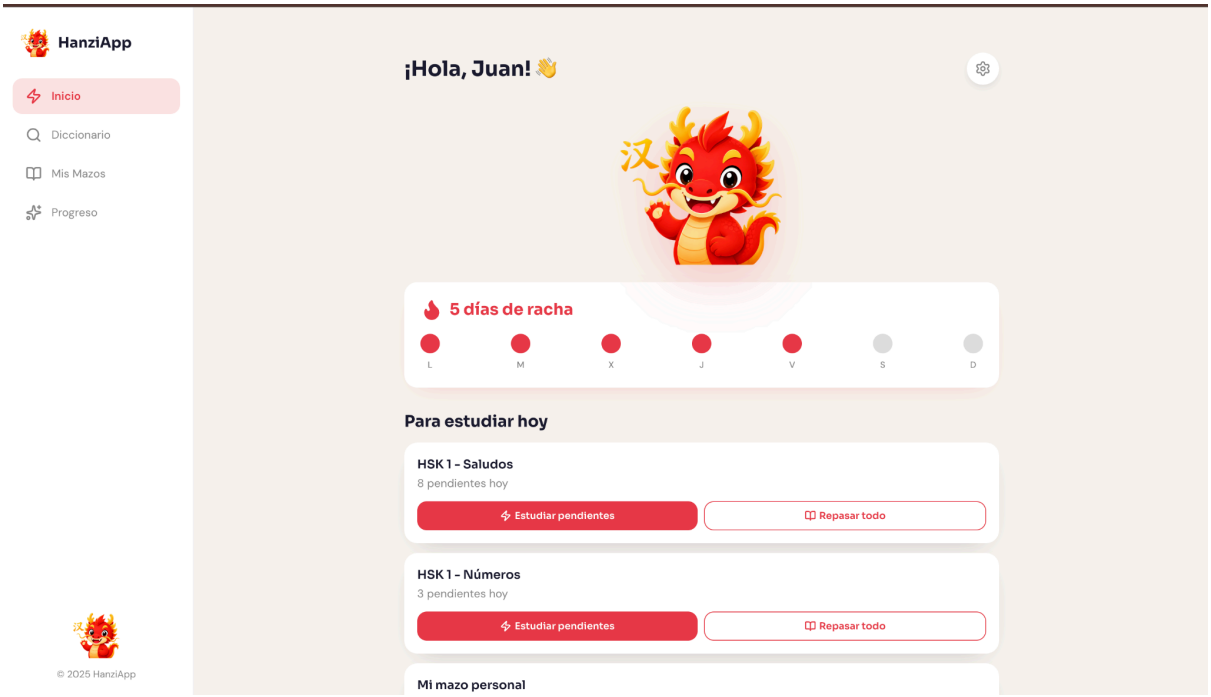
© 2025 HanziApp. Todos los derechos reservados.

Register and log in: Intuitive design for the user to log in or register.

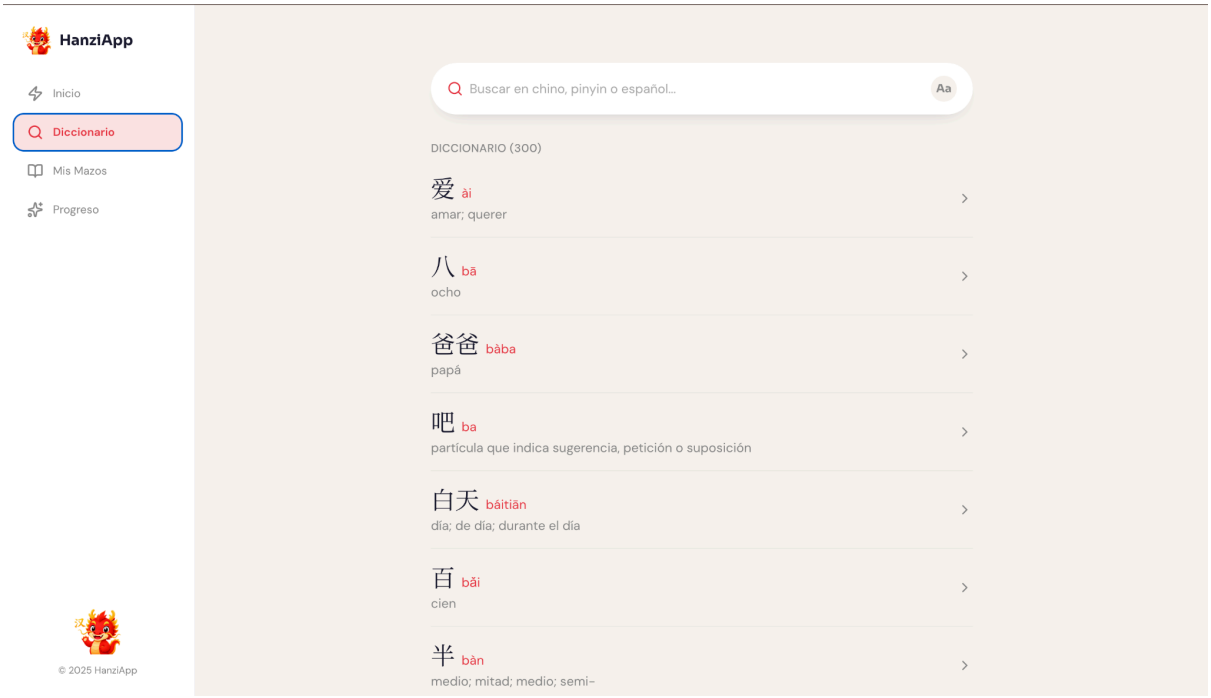




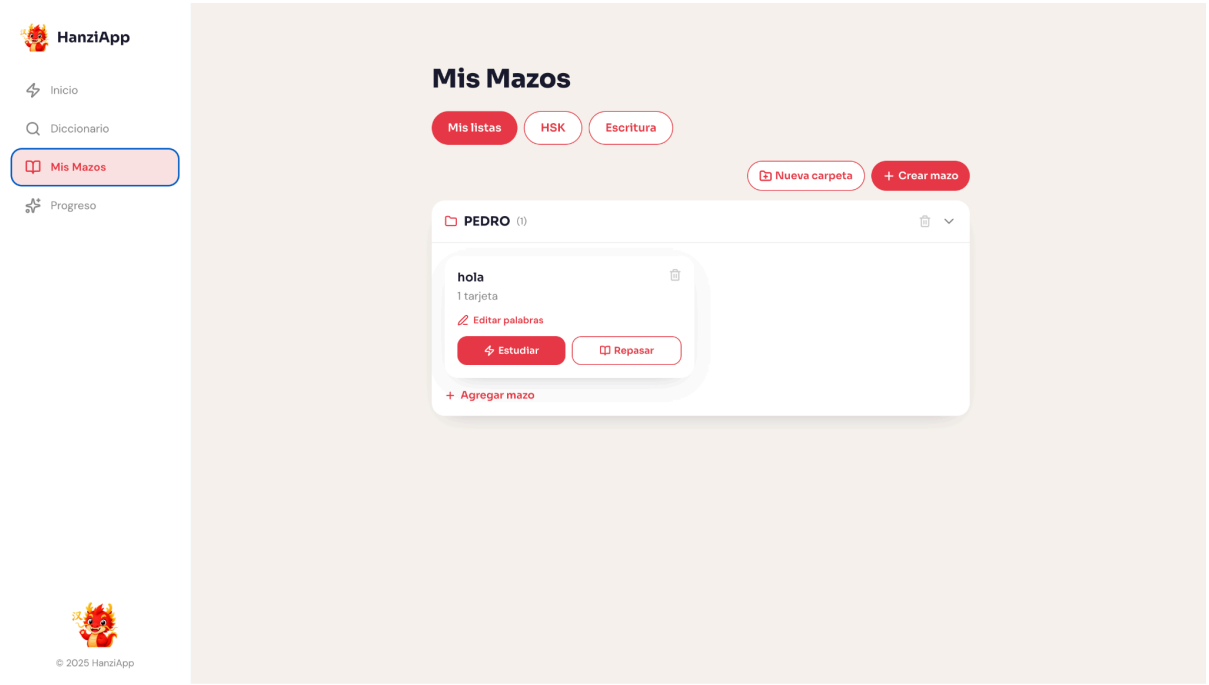
Main Page: Let the user look at his pending flashcards to study.

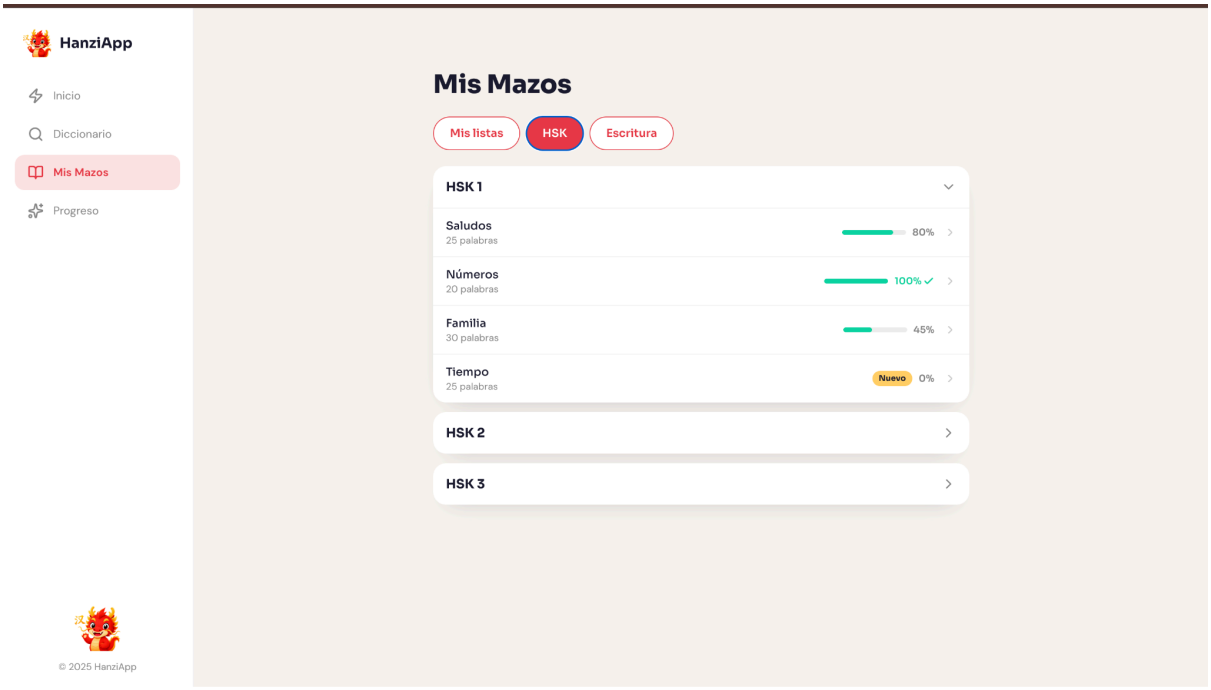


Dictionary: The user can look for words, meaning and add them to his decks.

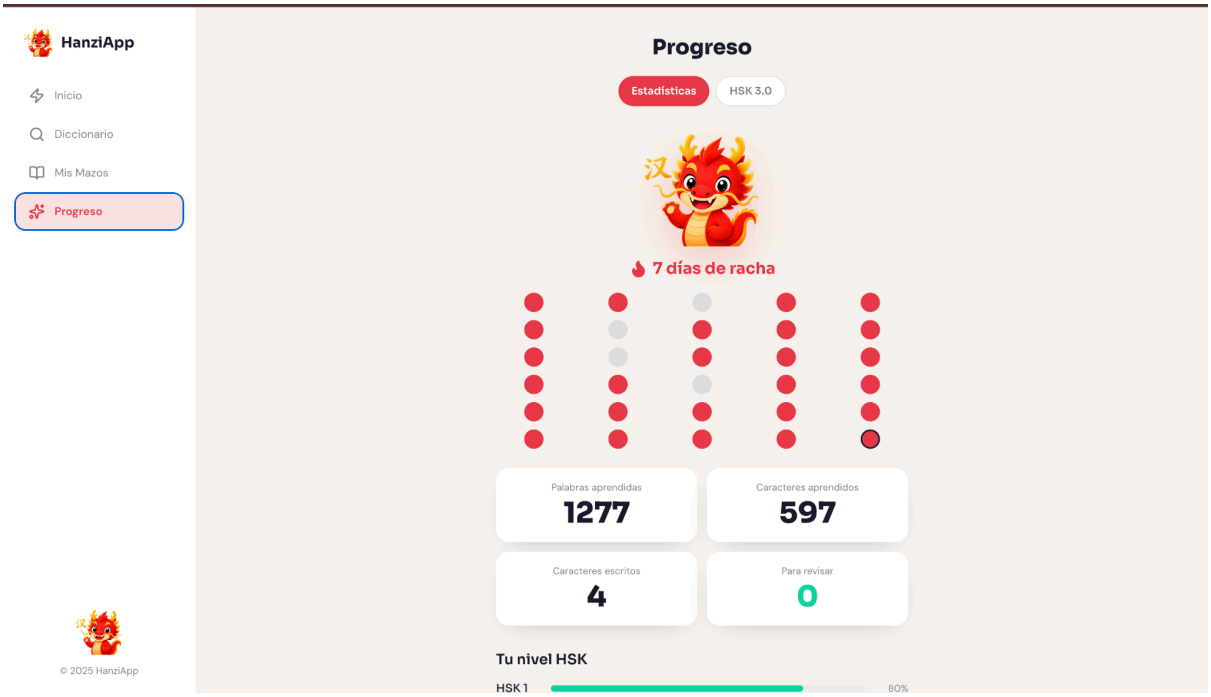


My decks: The user can control his lists and his decks.





Progress:. The user can look at his progress and his study streak.


HanziApp
Inicio
Diccionario
Mis Mazos
Progreso
© 2025 HanziApp
Progreso
Estadísticas
HSK 3.0
7 días de racha
Palabras aprendidas
1277
Caracteres aprendidos
597
Caracteres escritos
4
Para revisar
0
Tu nivel HSK
HSK 1
80%